PROJECT REPORT

Deep Learning-Based Intelligent OCR System for Handwritten and Complex Document Recognition

DeepOCR: preprocessing, text detection, deep-learning recognition, consensus voting, layout analysis and error correction in one system

---

October 2026

Abstract

Optical Character Recognition (OCR) works well on clean printed pages, but it still struggles with handwriting, noisy or skewed scans and documents that contain headings, tables and form fields. This project, **DeepOCR**, is an end-to-end OCR system for such documents. It cleans the page image, detects every text line, reads each line with a deep-learning recogniser, reconstructs the layout, corrects typical OCR errors and presents the result in a web application where a person can review and export it.

Three recognition approaches proposed for the project were built and compared. A CNN-BiLSTM-CTC baseline (2.39 million parameters) and a CRNN (2.78 million parameters) were implemented and trained from scratch on a laptop CPU, using 11,487 training lines from the IAM handwriting database and a synthetic printed-text dataset created for this work. As the advanced model, the Transformer-based TrOCR (61.6 million parameters) was used through transfer learning and fine-tuned on the same data, which lowered its character error rate (CER) on printed and degraded text from 20.37% to 2.59%. PaddleOCR serves as an industrial reference and supplies the DBNet text detector.

The main contribution beyond the proposal is a **consensus engine**: all five recognisers read each line, and their outputs are aligned and combined word by word with ROVER voting, weighted by each model's reliability and confidence. On 600 held-out IAM lines from unseen writers, the vote reaches **5.74% CER and 14.24% WER**, 15% fewer errors than the best single model. On 600 printed and degraded lines it reaches **2.35% CER and 5.86% WER**, a 46% reduction in word errors. Words that win no majority are flagged for human review. A robustness study shows that the system's deskewing step brings the baseline's CER at an 8° tilt back from 65.5% to 18.9%.

The system is delivered as a FastAPI service with a React web studio that shows live progress, page overlays, disputed-word alternatives, a review queue, engine comparison and the full experimental results. It is covered by 135 automated tests.

Keywords: optical character recognition, handwritten text recognition, CRNN, CTC, Transformer, TrOCR, ensemble voting, ROVER, document layout analysis, deep learning.

Contents

Abstract i

List of Figures vi

List of Tables viii

List of Abbreviations x

1. Introduction 1

    1.1 Background 1

    1.2 Motivation 1

    1.3 Problem Statement 2

    1.4 Objectives 2

    1.5 Scope of the Project 3

    1.6 Contributions 3

    1.7 Organisation of the Report 3

2. Literature Review 5

    2.1 Traditional OCR 5

    2.2 Recurrent Networks and CTC 5

    2.3 The CRNN Architecture 5

    2.4 Transformers for Text Recognition 6

    2.5 Text Detection 6

    2.6 Document Layout Analysis 6

    2.7 Combining Several Recognisers 7

    2.8 Post-OCR Error Correction 7

    2.9 Datasets 7

    2.10 Summary and Research Gap 7

3. System Analysis and Requirements 9

    3.1 Existing Systems and Their Limitations 9

3.2 Proposed System 9
3.3 Functional Requirements 9
3.4 Non-Functional Requirements 10
3.5 Hardware and Software Requirements 10
3.6 Feasibility Study 11
3.7 Use Cases 11
4. System Design 13
4.1 System Architecture 13
4.2 Deployment Architecture 14
4.3 Processing Sequence 14
4.4 Module Design 15
4.5 Recognition Model Design 16
4.6 Consensus Engine Design 18
4.7 Output Data Model 18
4.8 API Design 19
4.9 User Interface Design 20
5. Datasets and Preprocessing 21
5.1 IAM Handwriting Database 21
5.2 Synthetic Printed and Script Dataset 21
5.3 Data Splits 22
5.4 Label Normalisation and Character Set 22
5.5 Line Image Normalisation 22
5.6 Data Augmentation 22
5.7 Page Preprocessing 23
6. Implementation 25
6.1 Development Environment 25
6.2 Training the CTC Models 25

6.3 Fine-Tuning TrOCR 26
6.4 Text Detection and Line Extraction 26
6.5 Recognition Engines 26
6.6 Consensus Voting 27
6.7 Layout Reconstruction 27
6.8 Post-Processing and Error Correction 28
6.9 Backend Service 29
6.10 Web Application 29
6.11 Streamlit Research Interface 34
7. Experiments and Results 35
7.1 Evaluation Metrics 35
7.2 Experimental Setup 35
7.3 Training Results 35
7.4 Fine-Tuning Results 36
7.5 Model Comparison 37
7.6 Effect of Post-Processing 38
7.7 Consensus Voting 39
7.8 Robustness Study 40
7.9 Accuracy Versus Speed 41
7.10 End-to-End Results on a Document 42
7.11 Qualitative Results 42
7.12 Discussion 43
8. Testing 44
8.1 Testing Strategy 44
8.2 Automated Test Suites 44
8.3 Test Cases 45
8.4 Bugs Found and Fixed 45

9. Conclusion and Future Work 47
9.1 Conclusion 47
9.2 Objectives Achieved 47
9.3 Limitations 48
9.4 Future Work 48
References 49
Appendix A: Installation and Usage 52
Appendix B: Project Structure 53
Appendix C: Example API Response 54

List of Figures

Figure 3.1: Use-case diagram of DeepOCR 12

Figure 4.1: Processing pipeline of DeepOCR 13

Figure 4.2: Deployment architecture 14

Figure 4.3: Sequence diagram of an OCR job 15

Figure 4.4: Layer structure of the three recognition models with output shapes (channels × height × width) 16

Figure 4.5: Consensus voting on a real line: the slot where the models disagree 18

Figure 5.1: Example lines from the IAM test set (top three) and the synthetic test set (bottom three) with their ground truth 21

Figure 5.2: Augmentation operations applied to one IAM training line 23

Figure 5.3: Page preprocessing on the noisy sample: the 4.2° skew is detected and corrected, and noise and lighting are reduced 24

Figure 6.1: Studio start screen with the engine list showing measured error rates 29

Figure 6.2: Live progress of a consensus run: five models reading the eight detected lines 30

Figure 6.3: Results of the consensus engine on handwriting: agreement overlay and the alternatives for a disputed word 31

Figure 6.4: Review queue: each disputed word with one-click alternatives 31

Figure 6.5: Layout analysis of an application form: headings, checkboxes, a ruled table and fields 32

Figure 6.6: Compare page: five engines on the handwritten sample, scored word by word against the ground truth 33

Figure 6.7: Research page with the measured results 33

Figure 6.8: Results on a phone: document view (left) and recognised lines with disputed words (right) 34

Figure 7.1: Training of the CTC models: validation CER (left) and CTC loss (right) per epoch 36

Figure 7.2: Validation CER of TrOCR during fine-tuning.....36

Figure 7.3: CER (solid) and WER (light) of each engine on handwriting and printed text.....37

Figure 7.4: Best single model, consensus vote and oracle ceiling.....39

Figure 7.5: CER of each engine as each degradation gets stronger.....40

Figure 7.6: Accuracy versus speed of the engines (CPU, log scale) 41

List of Tables

Table 2.1: Summary of the main approaches reviewed 7

Table 3.1: Functional requirements 9

Table 3.2: Hardware used for development, training and evaluation 10

Table 3.3: Software stack 10

Table 4.1: Pipeline stages 13

Table 4.2: Main modules of the code base 15

Table 4.3: Recognition models used in the experiments 17

Table 4.4: Block kinds produced by layout analysis 19

Table 4.5: REST API of the service 19

Table 5.1: Dataset splits (number of text lines) 22

Table 5.2: Training-time augmentation 23

Table 6.1: Training settings 25

Table 7.1: Recognition results on the held-out test sets (600 lines each; CER and WER in %, lower is better) 37

Table 7.2: Line accuracy: percentage of test lines recognised exactly 38

Table 7.3: Effect of the error-correction stage on the CTC models (CER and WER in %) 38

Table 7.4: Consensus voting compared with the best single model (CER and WER in %; 2-fold cross-validated) 39

Table 7.5: CER (%) at the strongest level of each degradation (80 IAM test lines) 40

Table 7.6: End-to-end results on the handwritten sample page (8 lines, auto-correct off) 42

Table 7.7: Example predictions (errors in the CTC and TrOCR outputs are visible against the ground truth) 42

Table 8.1: Automated tests (all passing) 44

Table 8.2: Selected system test cases 45

Table 9.1: Objectives of the proposal and how they were met 47

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
BiLSTM	Bidirectional Long Short-Term Memory
BN	Batch Normalisation
CER	Character Error Rate
CLAHE	Contrast-Limited Adaptive Histogram Equalisation
CNN	Convolutional Neural Network
CPU / GPU	Central / Graphics Processing Unit
CRNN	Convolutional Recurrent Neural Network
CTC	Connectionist Temporal Classification
DBNet	Differentiable Binarization Network (text detector)
DeiT	Data-efficient Image Transformer
HTR	Handwritten Text Recognition
IAM	IAM Handwriting Database (Institut für Informatik und Angewandte Mathematik, Bern)
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
OCR	Optical Character Recognition
PP-OCR	PaddlePaddle OCR system
REST	Representational State Transfer
ROVER	Recognizer Output Voting Error Reduction
TrOCR	Transformer-based Optical Character Recognition
UI	User Interface
ViT	Vision Transformer
WER	Word Error Rate

1. Introduction

1.1 Background

Optical Character Recognition (OCR) is the conversion of text in images, such as scanned pages, photographs and PDF files, into machine-readable digital text. Once text is digital it can be searched, edited, indexed, translated and fed into other software. OCR therefore sits at the start of many digitisation workflows: archives of handwritten letters, university application forms, invoices, medical prescriptions and classroom notes all begin as images.

Early OCR systems compared each character with stored templates or hand-crafted features. They were accurate only for clean printed text in known fonts. Deep learning changed the field. Convolutional Neural Networks (CNNs) learn visual features directly from pixels, recurrent networks such as the Long Short-Term Memory (LSTM) model the order of characters, and the Connectionist Temporal Classification (CTC) loss lets a network learn from line images and their transcriptions without marking where each character is. More recently, Transformer models that use self-attention have reached the best published accuracy on handwriting.

Even so, a real document is more than a sequence of characters. It may be tilted, noisy or unevenly lit; it may mix print and handwriting; and it has structure such as headings, paragraphs, tables, checkboxes and signature fields. A useful OCR system must handle the image, find the text, read it, and preserve its structure. It should also show the user where the result might be wrong.

1.2 Motivation

Commercial OCR is widely available, but three gaps motivated this project. First, handwriting recognition remains error-prone, and users rarely know which words to double-check. Second, many tools return plain text and lose the layout of forms and tables. Third, cloud OCR services send documents to external servers, which is unsuitable for private records such as application forms. The aim was therefore a system that runs entirely on a local computer, explains its uncertainty, keeps the document structure and lets a person correct the output quickly.

The project also had an academic motivation: to implement the classical deep-learning OCR architectures, train them, and compare them fairly with a modern Transformer and an industrial engine on the same held-out data, using the standard metrics of the field.

1.3 Problem Statement

Existing OCR systems face difficulties when processing:

- handwritten documents and different writing styles;
- low-quality scans that are blurred, noisy, low in contrast or unevenly lit;
- skewed (rotated) page images;
- complex layouts with several text blocks, headings, tables and form fields;
- different fonts, and visually similar characters such as 0/O and 1/l.

The problem addressed in this project is to develop a deep-learning-based system that accurately extracts text from such documents, stays robust to variations in image quality, handwriting and layout, and gives the user a clear way to review and correct the result.

1.4 Objectives

The objectives were set in the project proposal:

1. To develop an OCR system for extracting text from document images.
2. To recognise handwritten as well as printed text.
3. To preprocess noisy, blurred and skewed document images.
4. To detect and extract text regions from complex documents.
5. To perform document layout analysis.
6. To compare different deep-learning models for text recognition.
7. To evaluate the system using standard OCR performance metrics.
8. To develop a user-friendly interface for uploading documents and viewing the extracted text.

Chapter 9 (Table 9.1) maps each objective to the part of the system that meets it.

1.5 Scope of the Project

The system reads English text, both printed and handwritten, from images (PNG, JPEG, TIFF, BMP, WEBP) and PDF files of up to 100 pages. Recognition works line by line. Detection, layout analysis and correction operate on whole pages. All training and inference run on a laptop CPU without a GPU, which shaped several design decisions (Section 6.2). Other scripts such as Hindi, mathematical notation, signature *verification* and learned layout models are outside the scope and are listed as future work.

1.6 Contributions

The main outcomes of the project are:

- **Two recognisers trained from scratch:** a CNN-BiLSTM-CTC baseline and a CRNN, both trained on CPU on real IAM handwriting and a purpose-built synthetic printed dataset.
- **Transfer learning:** the pretrained TrOCR Transformer was fine-tuned on the project data, reducing its printed-text CER from 20.37% to 2.59% while keeping its handwriting accuracy.
- **Consensus voting:** a ROVER-style word vote over five recognisers, with weights tuned by cross-validation, gives the lowest error of all engines and flags each disputed word.
- **A complete document pipeline:** deskewing, denoising, contrast enhancement, DBNet line detection, layout reconstruction (headings, tables, checkboxes, signature fields) and context-aware error correction.
- **Rigorous evaluation:** CER, WER, line accuracy and speed on held-out data; training curves; a robustness study under six kinds of degradation; and an end-to-end document test.
- **A web application:** a studio with live progress, overlays, disputed-word alternatives, a review queue and exports, plus pages for comparing engines and browsing the results.

1.7 Organisation of the Report

Chapter 2 reviews the literature on OCR, handwriting recognition, text detection, layout analysis and recogniser combination. Chapter 3 analyses the requirements and feasibility. Chapter 4 presents the system design and the model architectures. Chapter 5 describes the datasets and preprocessing. Chapter 6 explains the implementation.

Chapter 7 reports the experiments and results, and Chapter 8 the testing. Chapter 9 concludes the report and outlines future work. Appendices give installation steps, the project structure and an example API response.

2. Literature Review

2.1 Traditional OCR

Classical OCR pipelines binarise the page, segment it into lines, words and characters, and classify each character with template matching or hand-engineered features fed to a statistical classifier. Otsu's thresholding method [1] is still widely used for binarisation. The open-source Tesseract engine [2] followed this design for many years, adding adaptive classifiers and a dictionary. Such systems work well on clean, printed, single-column text, but character segmentation fails on cursive handwriting, where letters touch, and on degraded images. Tesseract later added an LSTM line recogniser, following the deep-learning approaches described below.

2.2 Recurrent Networks and CTC

Handwriting is naturally a sequence, so recurrent neural networks suit it. The LSTM [3] solves the vanishing-gradient problem of simple recurrent networks, and the bidirectional LSTM (BiLSTM) [4] reads the sequence in both directions so each position sees both left and right context. The key step that removed the need for character segmentation was Connectionist Temporal Classification (CTC) [5]. CTC adds a *blank* symbol and sums the probability of all alignments between the network's per-frame outputs and the target text, so a model can be trained from a line image and its transcription alone. Graves and Schmidhuber [6] applied multidimensional LSTMs with CTC to offline handwriting and won the ICDAR 2009 Arabic handwriting competition without any language-specific preprocessing.

2.3 The CRNN Architecture

Shi, Bai and Yao [7] proposed the Convolutional Recurrent Neural Network (CRNN), which combines a VGG-style CNN feature extractor, a BiLSTM sequence model and a CTC transcription layer in one network trained end to end. Its pooling layers shrink the image height faster than its width, so each column of the final feature map becomes one time step. CRNN handles text of any length, needs no character segmentation and is small, which made it the standard baseline for scene-text and handwriting recognition. Puigcerver [8] later showed that a CNN followed by one-dimensional

BiLSTM layers is as accurate as the more complex multidimensional LSTMs on IAM, supporting the CNN-BiLSTM-CTC design used for the baseline in this project.

2.4 Transformers for Text Recognition

The Transformer [9] replaces recurrence with self-attention, which relates every position of a sequence to every other position in one step. The Vision Transformer (ViT) [10] applied it to images by splitting them into 16 × 16 patches treated as tokens, and DeiT [11] made such models trainable with less data through distillation and strong augmentation. TrOCR [12] combines a pretrained image Transformer encoder with a pretrained text Transformer decoder. It is pretrained on hundreds of millions of synthetic printed lines and fine-tuned on handwriting. It reports CERs between about 2.9% and 4.2% on IAM depending on model size, the best published results at the time. Its drawbacks are size (61.6 million parameters even for the small model) and slow autoregressive decoding, which matter on a CPU.

2.5 Text Detection

Before lines can be recognised they must be found. Segmentation-based detectors predict for every pixel whether it belongs to text. EAST [13] predicts rotated boxes directly, and CRAFT [14] detects individual characters and links them. DBNet [15] introduced a *differentiable binarisation* module that learns the threshold map together with the segmentation, giving accurate, tight polygons at high speed. PP-OCR [16] packages a light DBNet detector, a direction classifier and a CRNN-style recogniser into an industrial system. This project uses its DBNet detector for every engine.

2.6 Document Layout Analysis

Layout analysis divides a page into regions such as titles, paragraphs, tables and figures, and determines their reading order. Classical methods use projection profiles and recursive X-Y cuts [17]. Learned methods train object detectors on large labelled corpora such as PubLayNet [18], and models such as LayoutLM [19] jointly use text, position and image features to understand forms and receipts. Learned layout models need large labelled datasets and GPUs. This project instead uses geometry-based rules on the detected lines, which run fast on a CPU and are transparent.

2.7 Combining Several Recognisers

Different recognisers make different mistakes, so combining them can reduce errors. The best-known method is ROVER (Recognizer Output Voting Error Reduction) [20], developed for speech recognition. It aligns the word sequences of several systems into a *word transition network* and decides each position by a (possibly confidence-weighted) vote. Lund and Ringger [21] showed that aligning and voting over several OCR engines also lowers OCR word error rates. The consensus engine in this project applies ROVER to five line recognisers of very different designs, with weights tuned by cross-validation.

2.8 Post-OCR Error Correction

OCR output can be improved after recognition. Kukich [22] surveys techniques for detecting and correcting non-word and real-word errors with dictionaries and edit distance. The Levenshtein distance [23], the minimum number of insertions, deletions and substitutions that turn one string into another, is the basis of both spelling correction and of the CER and WER metrics. OCR has its own error patterns, especially confusions between visually similar characters such as 0/O, 1/l and 5/S, which a character-level confusion model can fix in context.

2.9 Datasets

The IAM Handwriting Database [24] is the standard benchmark for English handwriting recognition. It contains forms written by 657 writers, segmented into lines with transcriptions, and an official *writer-independent* split in which no writer of the test set appears in training. For printed text, synthetic rendering with many fonts and degradations is common because it gives unlimited labelled data.

2.10 Summary and Research Gap

Table 2.1: Summary of the main approaches reviewed

Work	Approach	Strength	Limitation
Tesseract [2]	Segmentation + classifier, later LSTM	Fast, open source, good on clean print	Weak on handwriting and degraded scans
CTC [5], MDLSTM [6]	Recurrent network trained with CTC	No character segmentation needed	Sequential, slower to train
CRNN [7]	CNN + BiLSTM + CTC, end to end	Small, any text length, standard baseline	Limited language context

Work	Approach	Strength	Limitation
TrOCR [12]	ViT encoder + Transformer decoder	Best published accuracy on IAM	Large and slow on CPU, needs pretraining
DBNet [15], PP-OCR [16]	Segmentation-based detection	Accurate polygons, real-time	Recogniser tuned for print
LayoutLM [19]	Multimodal Transformer for layout	Understands complex forms	Needs labelled data and GPU
ROVER [20], [21]	Align and vote over several systems	Reduces errors of single systems	Cost grows with the number of systems

Most published systems focus on one stage, usually recognition on pre-cut lines. Few combine preprocessing, detection, recognition, layout and correction in one tool, compare classical and Transformer recognisers on identical held-out data, and tell the user which words are uncertain. This project addresses that gap. Its distinctive element is the use of word-level agreement between independent models, both to improve accuracy and to point the reviewer to the words most likely to be wrong.

3. System Analysis and Requirements

3.1 Existing Systems and Their Limitations

Three kinds of OCR tool are commonly available. **Desktop OCR engines** such as Tesseract are free and private but are tuned for clean printed text. **Cloud OCR services** are accurate on many document types, but they upload documents to external servers, charge per page and need an internet connection. **Research models** such as TrOCR are accurate but are released as model weights that need programming to use, and they work on single cropped lines rather than pages. None of these tools tells the user *which* words are unreliable in a way that guides correction, and most return plain text without structure.

3.2 Proposed System

DeepOCR is a local, end-to-end system. A user uploads a page or PDF in a browser. The server cleans the image, detects text lines, reads them with the selected engine (one model or a consensus of five), rebuilds the layout and corrects typical errors. The user then sees the page with coloured overlays, the recognised lines with confidence or agreement scores, disputed words with the other models' readings, and a review queue. Corrected text can be exported as Markdown, plain text or JSON with coordinates. Nothing leaves the computer.

3.3 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	Accept PDF, PNG, JPEG, TIFF, BMP and WEBP files, by upload, drag and drop, clipboard paste or camera.
FR2	Let the user choose the recognition engine, the image-cleanup preset and automatic correction.
FR3	Preprocess pages: deskew, denoise, enhance contrast and (optionally) binarise.
FR4	Detect text lines and return their polygons.
FR5	Recognise each line with CNN-BiLSTM, CRNN, TrOCR, fine-tuned TrOCR, PaddleOCR or the consensus of all five.
FR6	Reconstruct reading order, headings, key-value rows, tables, checkboxes and signature fields.
FR7	Correct typical OCR errors and report every change so it can be undone.
FR8	Show progress while processing (stage, lines read, models finished) and allow cancellation.
FR9	Display results with overlays, confidence or agreement, disputed-word alternatives, line editing and a review queue.

ID	Requirement
FR10	Export Markdown, plain text and JSON (with polygons and confidences).
FR11	Compare several engines on one document and score them against ground-truth text (CER, WER).
FR12	Present the evaluation results: model comparison, voting, training curves, robustness and samples.

3.4 Non-Functional Requirements

- **Accuracy:** report CER and WER on held-out data; the best engine should stay below 10% CER on handwriting from unseen writers.
- **Performance:** run on a laptop CPU; a one-page document with a single model should finish in a few seconds.
- **Reliability:** reject invalid, oversized or encrypted files with a clear message; process one document at a time and let other clients retry automatically; never crash on a bad page.
- **Privacy:** process and keep documents only in memory on the local machine; store nothing.
- **Usability:** a clean, consistent interface that works on desktop and phone screens, with keyboard access and visible focus.
- **Maintainability:** modular code, configuration files, reproducible scripts and automated tests.
- **Portability:** run on Windows (one-click launcher) and in Docker.

3.5 Hardware and Software Requirements

Table 3.2: Hardware used for development, training and evaluation

Component	Specification
Processor	Intel Core i7-1255U (10 cores, 12 threads)
Memory	16 GB RAM
GPU	None. All training and inference run on the CPU.
Storage	About 2 GB for datasets, checkpoints and dependencies
Operating system	Windows 11 (Docker image for Linux deployment)

Table 3.3: Software stack

Layer	Technology (version)
Language	Python 3.12; TypeScript 5

Layer	Technology (version)
Deep learning	PyTorch 2.4.1; Hugging Face Transformers 4.45.2; PaddlePaddle 2.6.2 / PaddleOCR 2.10
Computer vision	OpenCV 4.11; Pillow; PyMuPDF (PDF rendering)
Data and evaluation	NumPy; pandas; scikit-learn; Matplotlib
Text correction	pyspellchecker
Backend	FastAPI 0.116; Uvicorn
Frontend	React 18; Vite 6; lucide-react icons; react-markdown
Research interface	Streamlit
Testing	pytest; Vitest with Testing Library; Playwright (browser checks)

3.6 Feasibility Study

3.6.1 Technical Feasibility

All components rely on mature open-source libraries. The main technical risk was the lack of a GPU. It was handled by sizing the CTC models for CPU training (about two hours each), by using transfer learning instead of training a Transformer from scratch, and by fine-tuning TrOCR for only 300 steps (44 minutes). The project is therefore technically feasible on ordinary student hardware.

3.6.2 Economic Feasibility

The software is entirely open source, the IAM data is free for research, and the synthetic data is generated locally. No cloud services or paid APIs are used, so the running cost is zero.

3.6.3 Operational Feasibility

The system starts with one double-click on Windows and runs in a web browser. Users need no programming knowledge. Disputed words and review items guide them to the parts that need attention, which makes the tool practical for non-experts.

3.7 Use Cases

Figure 3.1 shows the main interactions between a user and the system.

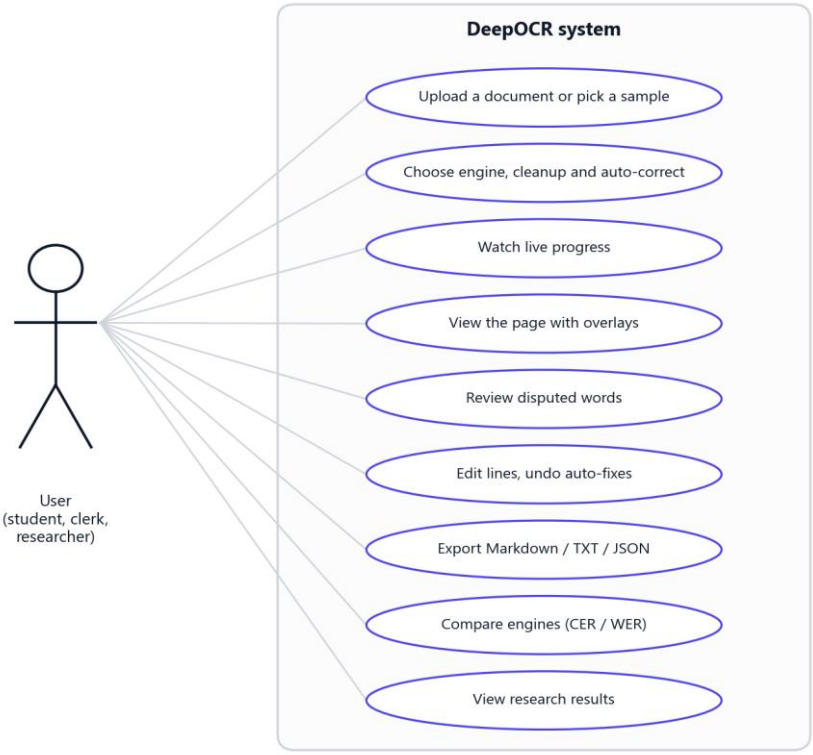


Figure 3.1: Use-case diagram of DeepOCR

4. System Design

4.1 System Architecture

The system follows the pipeline in the proposal, extended with a consensus stage. Figure 4.1 shows the eight stages and the code that implements each one, and Table 4.1 summarises their inputs and outputs.

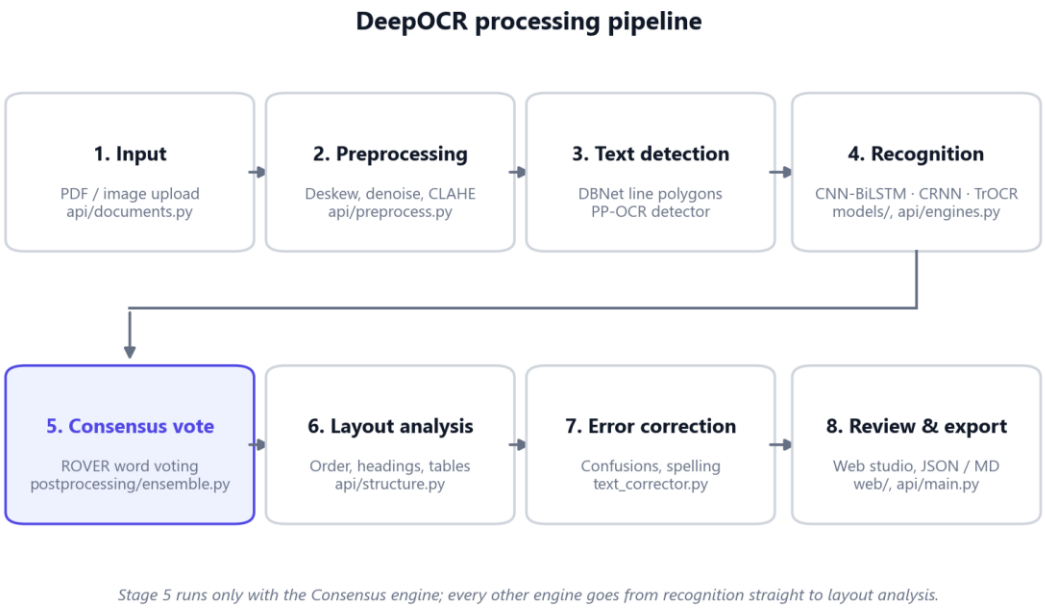


Figure 4.1: Processing pipeline of DeepOCR

Table 4.1: Pipeline stages

Stage	Input	Processing	Output
1. Input	Uploaded file	Validate type and size; render PDF pages; apply EXIF orientation	Page images
2. Preprocessing	Page image	Deskew, denoise, CLAHE contrast, optional binarisation	Clean page + stage previews
3. Text detection	Clean page	DBNet segmentation → line polygons → rectified crops	Line crops with polygons
4. Recognition	Line crops	Normalise each line; CTC decoding or Transformer decoding	Text + confidence per line
5. Consensus vote	Five readings per line	ROVER alignment and weighted word vote	Voted text + per-word votes
6. Layout analysis	Lines + page	Reading order, headings, tables, checkboxes, signature fields	Structured blocks
7. Error correction	Block text	Confusion fixes, spelling, normalisation	Corrected text + change list
8. Review & export	Blocks	Overlays, review queue, editing	Markdown / text / JSON

4.2 Deployment Architecture

The system is a client-server application (Figure 4.2). The React web app runs in the browser and talks to a FastAPI service over HTTP and JSON. The service keeps an engine registry that loads each model once, on first use, and reloads it automatically when a newer checkpoint is saved. Long OCR runs execute in a background job thread so the browser can follow their progress. Trained weights, pretrained models and evaluation results are read from local folders.

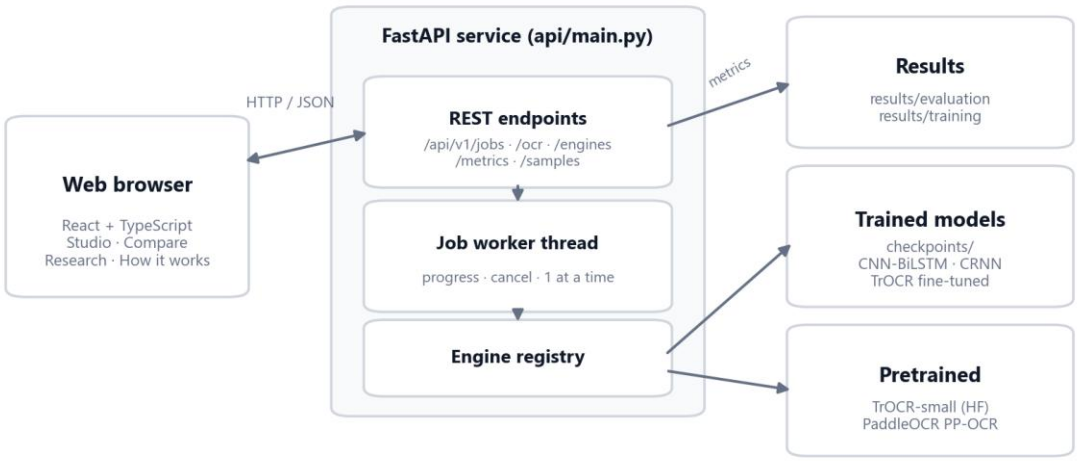


Figure 4.2: Deployment architecture

4.3 Processing Sequence

Figure 4.3 shows how a document is processed. The browser posts the file to `/api/v1/jobs` and immediately receives a job identifier. A worker thread runs the pipeline and reports its progress (current stage, lines read and, for the consensus engine, each model's progress). The browser polls the job every 350 ms to update the progress view and receives the full result when the job finishes. Cancelling sends `DELETE /api/v1/jobs/{id}`, which stops the worker at its next progress update.

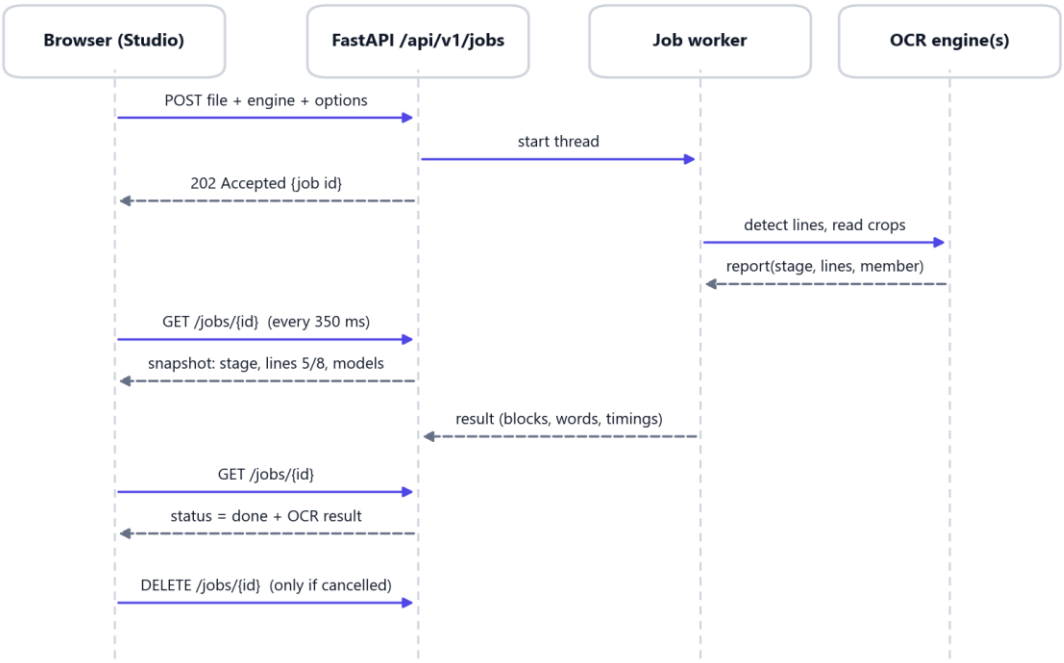


Figure 4.3: Sequence diagram of an OCR job

4.4 Module Design

Table 4.2: Main modules of the code base

Package	Responsibility
`api/`	FastAPI service: document decoding, page preprocessing, engine registry, jobs and progress, layout reconstruction, endpoints
`models/`	CNN-BiLSTM, CRNN and Transformer OCR definitions; recogniser wrappers for inference
`datasets/`	Dataset loaders, line normalisation, augmentation, splits
`training/`	Training loop, CTC utilities, checkpoints, early stopping
`evaluation/`	CER, WER and line-accuracy metrics
`preprocessing/`, `detection/`, `layout/`	Image operations, OpenCV fallback detector, research layout analyser
`postprocessing/`	Text corrector and ROVER consensus voting
`scripts/`	Data download and generation, training, fine-tuning, evaluation, robustness, ensemble tuning, samples
`web/`	React + TypeScript web studio
`app/`	Streamlit research interface (as specified in the proposal)
`tests/`, `api/tests/`	Automated tests

4.5 Recognition Model Design

Three recognisers were designed or adapted, as planned in the proposal: a baseline, a second CTC model and an advanced Transformer. Figure 4.4 compares their layers, and Table 4.3 their size and input.



Figure 4.4: Layer structure of the three recognition models with output shapes (channels × height × width)

4.5.1 Baseline: CNN-BiLSTM-CTC

The baseline follows the proposal's architecture. A grey line image of 32 × 512 pixels passes through four convolution blocks (3 × 3 convolution, batch normalisation and ReLU) with 32, 64, 128 and 256 channels. The first two blocks halve both height and width. The last two halve only the height, so the feature map keeps 128 columns. The 256 × 2 feature map of each column is flattened into a 512-dimensional vector, giving a sequence of 128 time steps. A two-layer BiLSTM with 192 units per direction reads the sequence, and a linear layer outputs a probability for each of 83 classes (82 characters and the CTC blank) at every step. Training uses the CTC loss, and inference

uses greedy CTC decoding: take the most likely class per step, merge repeats and remove blanks.

4.5.2 Model 2: CRNN

The CRNN follows Shi et al. [7]. Its CNN has seven convolution layers in a VGG pattern (32, 64, 128, 128, 256, 256, 256 channels, each with batch normalisation and ReLU). Pooling uses 2 × 2 windows first and then 2 × 1 windows, so height shrinks faster than width, and a final 2 × 2 convolution without padding collapses the height to one. This yields 127 time steps of 256 features. Two bidirectional LSTM layers, each followed by a linear projection, produce the 83-class outputs for CTC. The channel widths are half of those in the paper so the model can be trained on a CPU in about two hours; the topology is unchanged.

4.5.3 Advanced Model: TrOCR with Transfer Learning

The advanced model is TrOCR-small [12]. The line image is resized to 384 × 384 RGB and split into 576 patches of 16 × 16 pixels. A 12-layer DeiT encoder turns them into visual features. A 6-layer Transformer decoder attends to these features and generates the text one word-piece at a time. Training a Transformer of this size from scratch requires far more data and compute than a CPU allows, so, as the proposal suggests, transfer learning was used. The released `microsoft/trocr-small-handwritten` model was evaluated as it is, then fine-tuned for 300 steps on the project's IAM and synthetic training data (Section 6.3). A smaller custom ViT encoder-decoder is also implemented in `models/transformer_ocr.py` for completeness, but it was not used in the experiments.

Table 4.3: Recognition models used in the experiments

Model	Parameters	Input	Decoder	Trained by
CNN-BiLSTM	2.39 M	32 × 512 grey	CTC greedy	This project (from scratch)
CRNN	2.78 M	32 × 512 grey	CTC greedy	This project (from scratch)
TrOCR-small	61.6 M	384 × 384 RGB	Autoregressive	Microsoft (pretrained)
TrOCR-small, fine-tuned	61.6 M	384 × 384 RGB	Autoregressive	This project (300 steps)
PaddleOCR PP-OCR	Not published	48 px high	CTC	Baidu (reference engine)

4.6 Consensus Engine Design

Because the five recognisers differ in architecture, training data and input size, they tend to make *different* mistakes on the same word. If a word is read the same way by several independent models, it is more likely to be right. The consensus engine exploits this.

For each detected line, every model produces a hypothesis: its text and its line confidence. Each hypothesis receives a vote weight equal to the model's reliability weight multiplied by its confidence. The hypotheses are aligned word by word into slots (the word transition network of ROVER), and in each slot the word with the largest total vote wins. Figure 4.5 shows a real example from the handwritten test page. Two models read “fan” and three read “Pan”; the correct “Pan” wins with 66% of the vote.

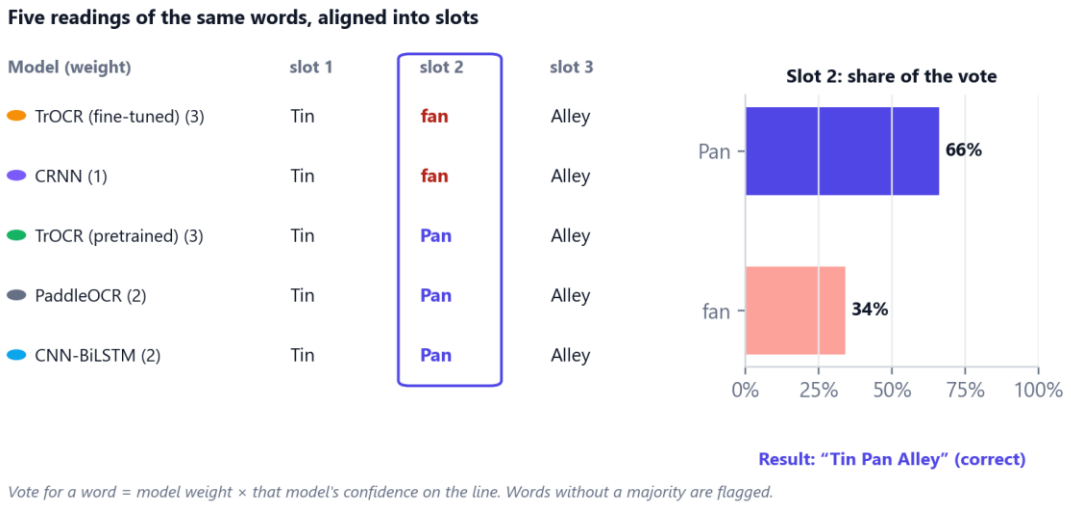


Figure 4.5: Consensus voting on a real line: the slot where the models disagree

Each output word carries its **support**, the share of the total vote it won, and its **alternatives** with their shares. A word with less than 50% support had no majority and is marked *disputed*. The web app underlines it and offers the alternatives with one click. The reliability weights were tuned on held-out data with cross-validation (Section 6.6): TrOCR fine-tuned 3, TrOCR 3, PaddleOCR 2, CNN-BiLSTM 2 and CRNN 1.

4.7 Output Data Model

The OCR result of each page is a list of *blocks* in reading order. Table 4.4 lists the block kinds. Every block has its text, a Markdown rendering, a polygon on the page, a

confidence, a review flag, the list of automatic corrections and, for the consensus engine, the per-word votes.

Table 4.4: Block kinds produced by layout analysis

Kind	Detected from	Markdown output
title / heading	Line height and stroke width clearly larger than the body text	# Title, ## Heading
text	Ordinary lines grouped into reading order	Plain line
table	Ruled grid lines on the page, or aligned cells in consecutive rows	Markdown table
checkbox	Small square boxes left of a line; filled or empty	[x] / [ ] / [?] + label
signature	A printed label such as “Signature” next to a blank or handwritten area	Label + [Signature]
illegible	Line confidence below 0.45	[Illegible] (raw text kept for review)

4.8 API Design

Table 4.5: REST API of the service

Method	Path	Purpose
POST	/api/v1/jobs	Start OCR in the background; returns a job id (202). Fields: file, engine, preprocess, correct
GET	/api/v1/jobs/{id}	Progress snapshot (stage, page, lines, per-model progress, stage times) and, when done, the result
DELETE	/api/v1/jobs/{id}	Cancel a running job
POST	/api/v1/ocr	Synchronous OCR (same fields, waits for the result)
GET	/api/v1/engines	Engines with availability, model details and measured metrics
GET	/api/v1/metrics	Evaluation summary, voting results, training histories, robustness, sample predictions
GET	/api/v1/samples	Built-in sample documents (with ground-truth text where available)
GET	/api/v1/health/live, /ready	Liveness and readiness checks
GET	/api/docs	Interactive OpenAPI documentation

Errors use standard status codes: 413 when size, page or pixel limits are exceeded; 415 for an unsupported file type; 422 for an invalid, encrypted or unreadable file or option; 429 when another document is being processed (the web app retries automatically); and 503 when an engine is unavailable.

4.9 User Interface Design

The interface follows a calm, light design: a neutral background, white cards and one accent colour, with colour reserved for meaning. Green, amber and red show confidence or agreement; a fixed colour identifies each model; and a fixed colour marks each layout kind. The web app has four pages: **Studio** (upload, live progress and the results workspace), **Compare** (several engines side by side), **Research** (all experimental results) and **How it works** (the pipeline). Their screens are shown in Section 6.10.

5. Datasets and Preprocessing

5.1 IAM Handwriting Database

Real handwriting comes from the IAM Handwriting Database [24], using its line-level images and transcriptions from the Teklia IAM-line export. It contains English text written by 657 different people. The official writer-independent split is used, so the test set's writers never appear in training or validation. This makes the test score a fair estimate of performance on a *new* person's handwriting.

5.2 Synthetic Printed and Script Dataset

IAM contains only handwriting, but real documents also contain printed text, numbers, e-mail addresses and form labels. A synthetic dataset was therefore generated (`scripts/prepare_dataset.py`). Lines are rendered with 19 printed fonts and 13 handwriting-style fonts. The text comes from IAM *training* transcriptions (never validation or test) and from document templates such as dates, amounts, phone numbers, e-mail addresses and form labels. Each line is then degraded with uneven lighting, ink bleed or fading, skew, blur, low contrast, Gaussian and salt-and-pepper noise, JPEG compression, or inversion (light text on a dark background).

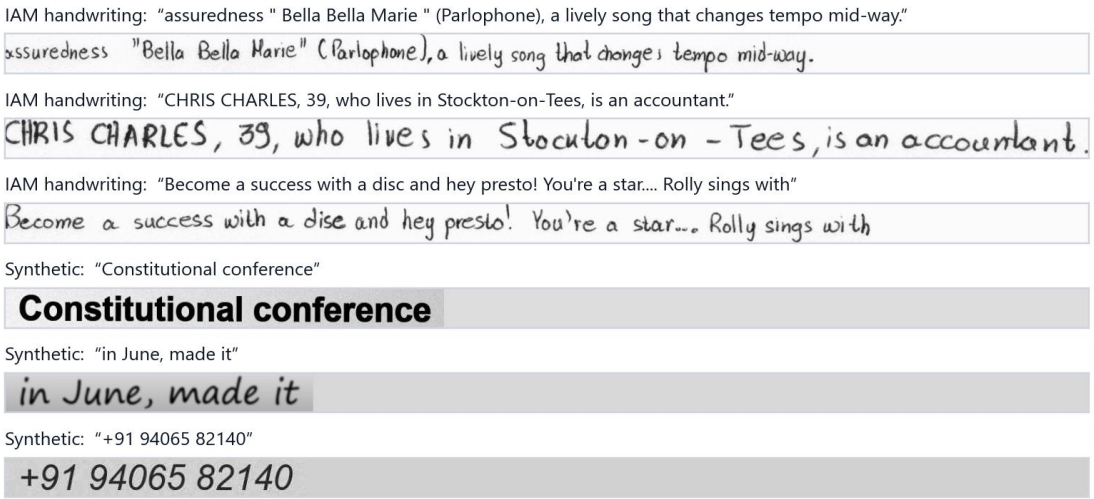


Figure 5.1: Example lines from the IAM test set (top three) and the synthetic test set (bottom three) with their ground truth

5.3 Data Splits

Table 5.1: Dataset splits (number of text lines)

Dataset	Train	Validation	Test	Total
IAM handwriting	6,482	976	2,915	10,373
Synthetic printed + script	5,005	520	975	6,500
Combined training set	11,487	1,496	-	-

The models are trained on both training sets together and selected on both validation sets. Test sets are used only for the final evaluation. For the comparison, 600 test lines are drawn from each test set with a fixed random seed (42), and every engine is scored on exactly the same lines.

5.4 Label Normalisation and Character Set

IAM transcriptions are tokenised, with spaces around punctuation (“mid-way .”, “we 've”). Printed text is not. To score every model fairly, all transcriptions are mapped to natural spacing (`normalize_transcription`): spaces before closing punctuation and after opening brackets are removed, and contractions are rejoined. The character set has 82 symbols: space, letters, digits and common punctuation including $, % and @ for document text. With the CTC blank, the CTC models therefore have 83 output classes.

5.5 Line Image Normalisation

Line images differ in polarity, brightness and size. One shared function (`normalize_line`) is used during training, evaluation and inference, so a model always sees the same kind of input. It converts the image to greyscale and makes the ink bright on a dark background (the background, which dominates a line, is estimated with the median). It then subtracts the background level, stretches the contrast so the 99.5th percentile becomes white, resizes the line to a height of 32 pixels keeping its aspect ratio, and pads or fits it to a width of 512 pixels.

5.6 Data Augmentation

To make the models robust to real-world variation, training lines are augmented on the fly with the operations listed in the proposal (Table 5.2, Figure 5.2). The test data is never augmented.

Table 5.2: Training-time augmentation

Operation	Setting
Random rotation	Up to ±1.5°
Random scaling (without cropping)	0.8 to 1.15 ×
Gaussian noise	Standard deviation 0.04
Blur	Probability 0.3, kernel up to 2 px
Brightness and contrast	Factors 0.6 to 1.4
Perspective transformation	Corner displacement up to 8%

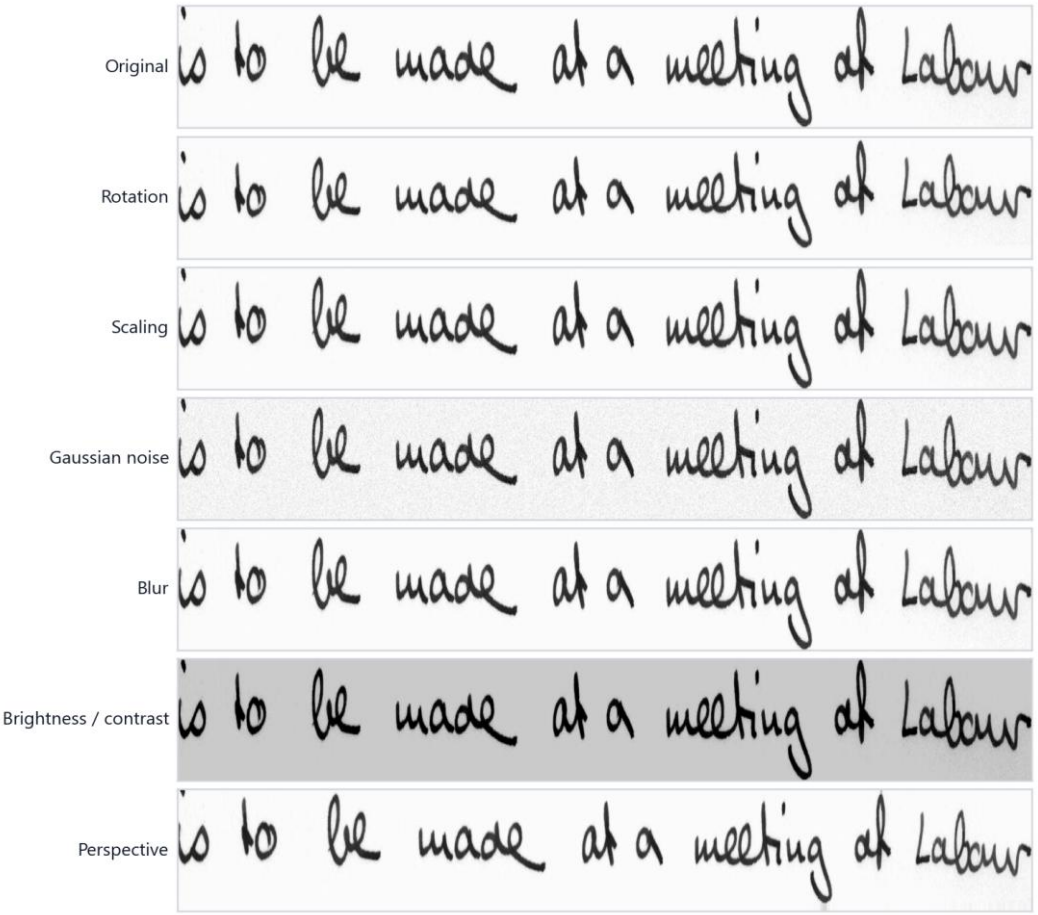


Figure 5.2: Augmentation operations applied to one IAM training line

5.7 Page Preprocessing

Before detection, the whole page can be cleaned with four operations, applied in a fixed order. The web app groups them into presets: *Auto* (deskew and contrast), *Scan* (adds denoising), *Photo* (adds binarisation) and *Off*.

- **Deskew:** the skew angle is estimated with a projection profile. A text mask is computed (flat-field correction followed by Otsu thresholding, which is robust

to shadows and speckle). The mask is rotated over −12° to +12°, first in 1° steps and then in 0.25° steps around the best angle. The angle that gives the sharpest row profile (highest variance of the row sums) aligns the text lines. The page is rotated only if the angle is at least 0.3°.

- **Denoise:** Gaussian and median filtering remove sensor noise and speckle.
- **Contrast:** CLAHE [25] raises local contrast so faint ink becomes visible without over-amplifying noise.
- **Binarise:** adaptive (local-mean) thresholding evens out uneven lighting in phone photographs.

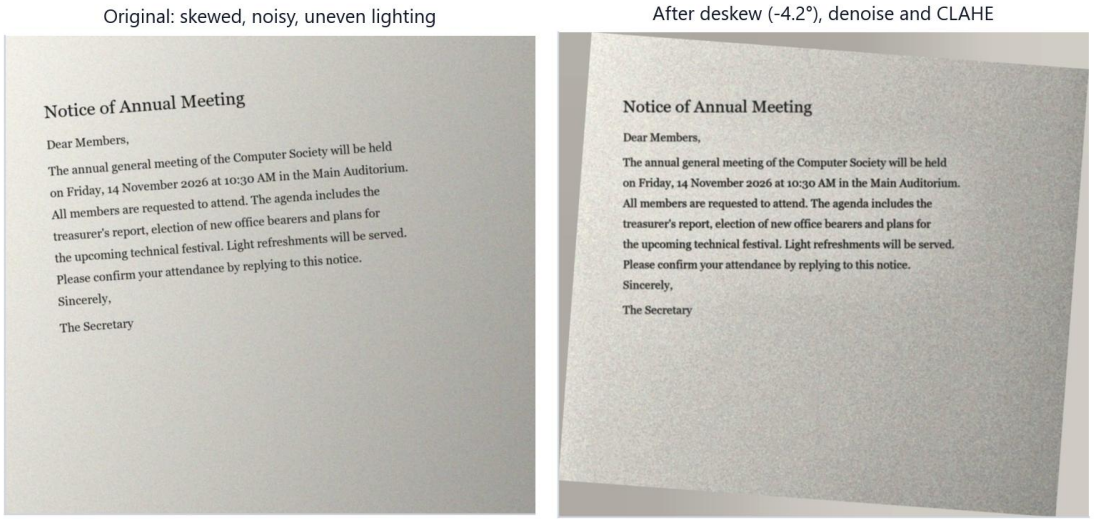


Figure 5.3: Page preprocessing on the noisy sample: the 4.2° skew is detected and corrected, and noise and lighting are reduced

6. Implementation

6.1 Development Environment

The system is written in Python (backend and machine learning) and TypeScript (web app). The proposal's technology stack (Python, PyTorch, OpenCV, PIL, NumPy, pandas, Matplotlib, scikit-learn and Streamlit) is used throughout. The following were added: Hugging Face Transformers for TrOCR, PaddleOCR for detection and as a reference engine, FastAPI for the service and React for the web studio. Configuration is kept in YAML files under `configs/`, and every experiment can be reproduced with the scripts listed in Appendix A.

6.2 Training the CTC Models

Both CTC models were trained with the same procedure (`scripts/train.py`, `training/`). Batches of 32 lines from the combined training set are augmented, normalised and passed through the network. The CTC loss is minimised with Adam, and the learning rate follows a cosine schedule. Gradients are clipped at a norm of 5 to keep the recurrent layers stable. After every epoch the model is evaluated on the validation sets, and the checkpoint with the lowest validation CER is saved. Training stops early if the CER does not improve for 8 epochs or when a wall-clock budget is reached. The checkpoint stores the architecture, character set and input size, so inference rebuilds exactly the trained model.

Because no GPU was available, the two models were trained side by side on the CPU, for 11 and 9 epochs respectively (about two hours each). Table 6.1 lists the settings.

Table 6.1: Training settings

Setting	CNN-BiLSTM	CRNN	TrOCR fine-tuning
Parameters	2,392,979	2,780,115	61,596,672
Training lines	11,487 (IAM + synthetic)	11,487 (IAM + synthetic)	IAM + synthetic, 50/50 batches
Optimiser	Adam, weight decay 1e-4	Adam, weight decay 1e-4	AdamW, weight decay 0.01
Learning rate	1e-3, cosine schedule	1e-3, cosine schedule	3e-5, linear warm-up and decay
Batch size	32	32	8
Length of training	11 epochs	9 epochs	300 steps
Wall-clock time (CPU)	124 min	130 min	44 min

Setting	CNN-BiLSTM	CRNN	TrOCR fine-tuning
Best validation CER	11.8%	12.2%	3.4% (mean of both sets)
Gradient clipping (norm)	5.0	5.0	1.0
Model selection	Lowest validation CER	Lowest validation CER	Lowest validation CER

6.3 Fine-Tuning TrOCR

The pretrained TrOCR-small model reads handwriting well but performs poorly on printed and degraded lines (20.37% test CER), because it was fine-tuned by its authors only on IAM. It was therefore fine-tuned on the project's data (`scripts/finetune_trocr.py`). Each batch of 8 lines is drawn half from IAM and half from the synthetic set and lightly augmented. Training uses AdamW with a learning rate of 3×10^{-5}, a short linear warm-up and linear decay. The validation CER on both sets was measured every 75 steps, and the best weights were kept.

One implementation detail mattered. TrOCR's decoder starts from token id 2 (its generation configuration), and its training targets must not contain the start token. An early attempt used the wrong start token, and the loss stayed near 11.7. After this was corrected, the loss fell steadily (0.95 after 75 steps, 0.56 after 300). Fine-tuning took 44 minutes on the CPU.

6.4 Text Detection and Line Extraction

Text lines are detected with the DBNet detector of PaddleOCR. Its box threshold was lowered from the default to 0.5 so that faint handwritten lines are not missed. Each detected polygon is cut out with a perspective transform, which straightens slightly tilted lines, and passed to the recogniser. If PaddleOCR is not installed, an OpenCV detector based on maximally stable extremal regions (MSER) is used as a fallback. On Windows, PyTorch must be imported before PaddlePaddle to avoid a DLL conflict; the package import order enforces this.

6.5 Recognition Engines

An engine registry (`api/engines.py`) exposes six engines: `cnn_bilstm`, `crnn`, `trocr`, `trocr_ft`, `paddle` and `ensemble`. Each loads lazily on first use. The CTC and TrOCR engines share the DBNet detector and read the line crops in chunks of four, reporting progress after each chunk. Line confidence is the mean of the per-character probabilities

(CTC) or of the generated tokens' probabilities (TrOCR). The registry checks the modification time of each checkpoint and reloads a model automatically when it has been retrained.

6.6 Consensus Voting

The consensus engine runs all five recognisers on the same line crops and combines their readings with the algorithm below (`postprocessing/ensemble.py`).

Algorithm 1: ROVER word voting for one text line

```
Input : hypotheses H = {(text_m, model_m, v_m)}
        vote v_m = weight_m × max(confidence_m, 0.01)
Output: voted line text; for each word its support and alternatives

1  sort H by vote, highest first; pivot ← H[0]
2  slots ← one slot per word of the pivot, each holding the pivot's vote
3  for each remaining hypothesis h in H:
4      align the words of h to the slots by edit distance, where
         substitution cost = 1.5 × normalised character distance
         (so near-misses such as "assuredness" / "assurances" share a slot)
5      add h's vote to its word in the aligned slot
         (or to EMPTY where h has no word)
6      for a word of h that matches no slot: open a new slot (others vote EMPTY)
7  for each slot:
8      winner ← word with the highest total vote
         (ties go to the word of the stronger model)
9      if winner ≠ EMPTY: output winner with support = vote(winner) / total vote
         and alternatives = the other words with their share of the vote
10 return the joined winners and the per-word votes
```

The model weights were tuned by grid search (`scripts/ensemble_eval.py`). The fine-tuned TrOCR was fixed at weight 3, and each other model was given a weight from {0, 1, 2, 3}, which gives 256 combinations. To avoid measuring on the lines used for tuning, 2-fold cross-validation was used: weights tuned on one half of the 1,200 test lines (600 IAM and 600 synthetic) were scored on the other half, and the halves were then swapped. The reported consensus scores are these cross-validated scores. The final weights, fitted on all lines, are TrOCR fine-tuned 3, TrOCR 3, PaddleOCR 2, CNN-BiLSTM 2 and CRNN 1. The region's confidence is the vote-weighted mean of the members' confidences.

6.7 Layout Reconstruction

Layout analysis (`api/structure.py`) works on the detected lines and the page image:

- **Line grouping:** text fragments are grouped into lines using rotated geometry (each fragment's baseline angle), so tilted lines are not merged, and the lines are sorted into reading order, column by column.
- **Headings:** a line whose height and stroke width are clearly larger than the median body line becomes a title or heading.
- **Key-value rows:** rows such as “Name: Ananya Iyer” stay text rows rather than being forced into tables.
- **Tables:** ruled tables are found from long horizontal and vertical lines in the image, and their cells are filled with the text inside. Unruled tables are recognised only when consecutive rows contain repeated, clearly separated cells.
- **Checkboxes:** small square outlines to the left of a line are detected, and their fill ratio decides between checked, unchecked and uncertain.
- **Signature fields:** a printed label (“Signature”, “Signed by”, “Authorized signatory”) marks a signature field. This identifies the field; it does not verify a signature.
- **Low confidence:** lines with confidence below 0.45 become *illegible* blocks and are added to the review queue with their raw text.

6.8 Post-Processing and Error Correction

The text corrector (`postprocessing/text_corrector.py`) applies, in order:

1. Unicode normalisation and conversion of typographic punctuation to ASCII.
2. Context-aware character-confusion fixes. Digits inside words become letters (“0ffice” → “Office”), letters inside numbers become digits (“2O24” → “2024”), and an “@” misread as O, 0 or Q in an e-mail address is restored. Pure numbers, codes and amounts are never changed.
3. Whitespace normalisation.
4. Word-level spelling correction with pyspellchecker, limited to alphabetic words of three or more letters at edit distance 1. ALL-CAPS acronyms are skipped. Capitalised words (likely names) accept only letter swaps, so a name such as “Iyer” is not turned into a dictionary word.

Every change is returned as a correction record (original word, corrected word, kind and position). The web app lists the changes and lets the user undo each one.

6.9 Backend Service

The FastAPI service (`api/main.py`) validates every upload before processing: at most 100 MB, 100 pages, 10,000 pixels per side, 40 megapixels per page and 100 megapixels in total. These limits protect the machine from oversized or malicious files. PDFs are rendered with PyMuPDF at up to twice their native resolution. Only one document is processed at a time. A second request receives HTTP 429, and the web app retries automatically. Jobs and results live only in memory and expire after 10 minutes (at most 32 are kept). The service also serves the built web app, so a single process provides the whole system.

6.10 Web Application

The web studio is a React and TypeScript single-page application built with Vite. Its main screens are shown below.

6.10.1 Studio: New Extraction

The start screen (Figure 6.1) explains the system and offers the extraction form: the document (drop, browse, paste or camera), the engine list with each engine's measured error on handwriting and print and its speed, and the options (cleanup preset and auto-correct). Four sample documents run with one click.

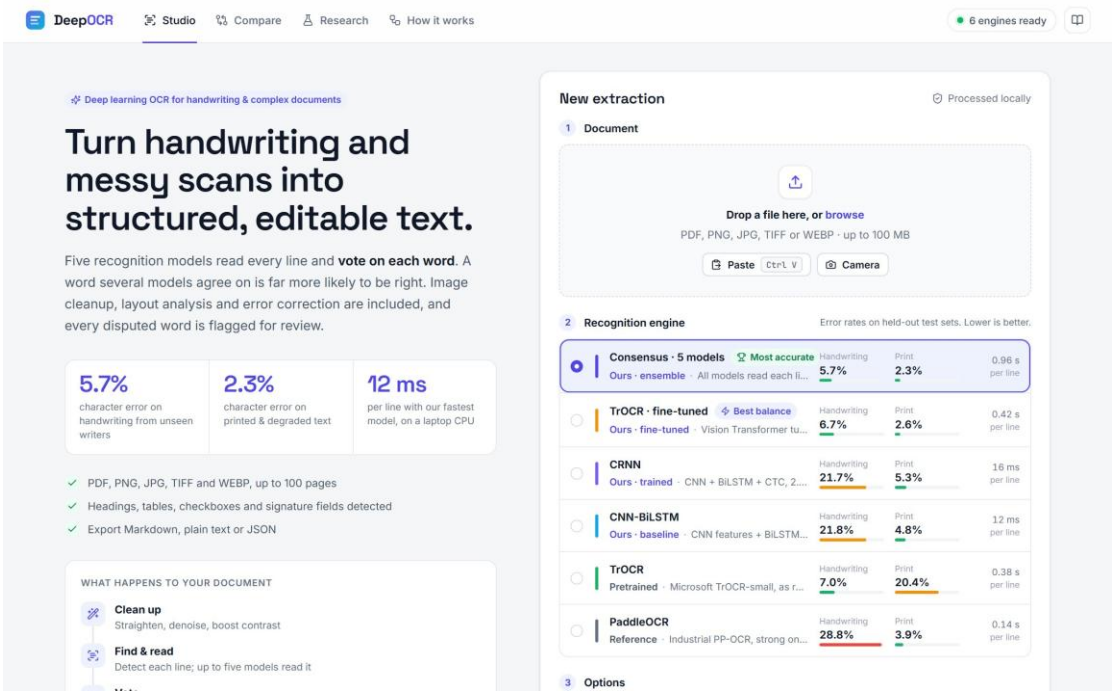


Figure 6.1: Studio start screen with the engine list showing measured error rates

6.10.2 Live Progress

While a document is processed, the progress card (Figure 6.2) shows each pipeline stage with its duration. A scanning line moves over the page preview. With the consensus engine, a chip for each model shows how many lines it has read.

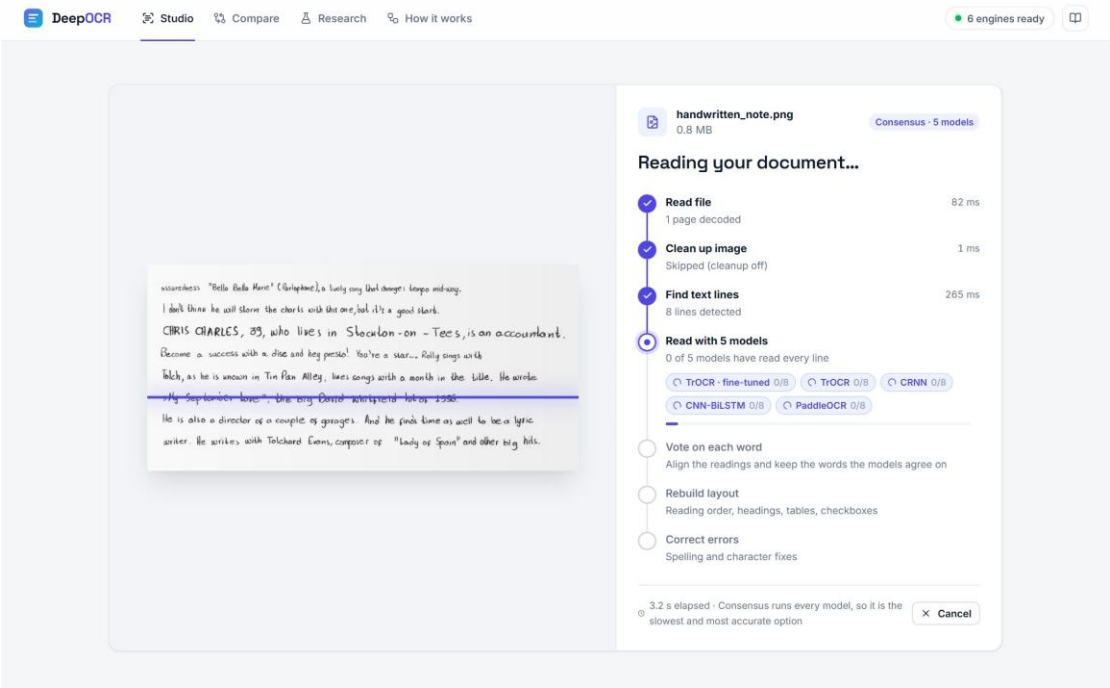


Figure 6.2: Live progress of a consensus run: five models reading the eight detected lines

6.10.3 Results Workspace

The results workspace (Figure 6.3) has a summary strip (agreement or confidence, lines, words, disputed words or review items, models and time), a bar listing the automatic fixes, a page rail, the document viewer and the text panel. The viewer overlays the detected blocks, coloured by layout kind, by agreement between models, or by confidence. A before/after slider compares the page with its cleaned version. In the Lines view, disputed words are underlined; clicking one shows the other models' readings and their share of the vote, and one click replaces the word.

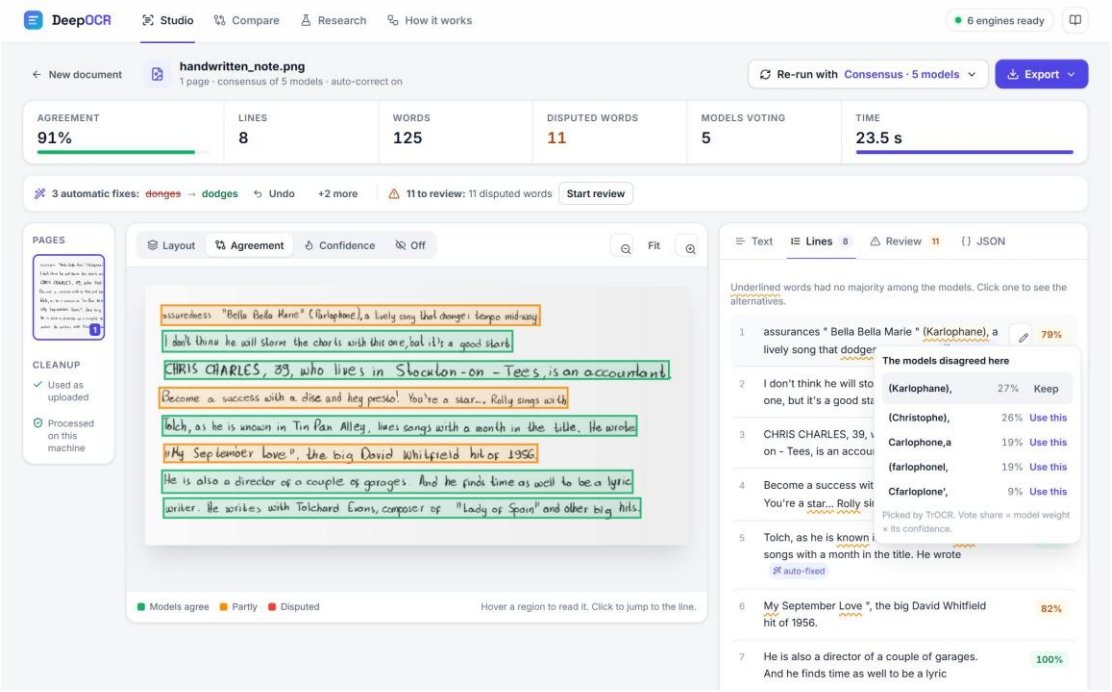


Figure 6.3: Results of the consensus engine on handwriting: agreement overlay and the alternatives for a disputed word

The Review tab (Figure 6.4) collects everything that needs a human check: disputed words, unclear lines, tables, checkboxes and signature fields. Each item can be located on the page and confirmed or corrected. The Text tab shows a formatted preview and an editable Markdown view, and the JSON tab the full structured result.

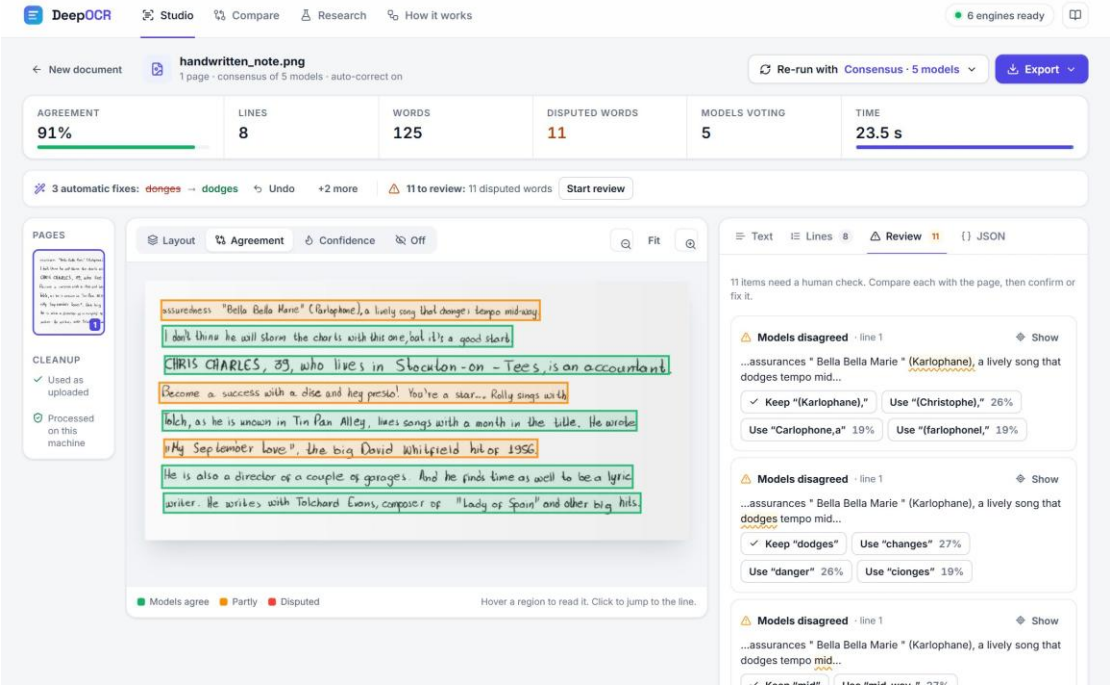


Figure 6.4: Review queue: each disputed word with one-click alternatives

Figure 6.5 shows a printed application form processed with PaddleOCR. The layout overlay marks the title, text rows, checkboxes (amber), the ruled table (green) and the signature field, and the preview shows them as Markdown with checkbox symbols and a table.

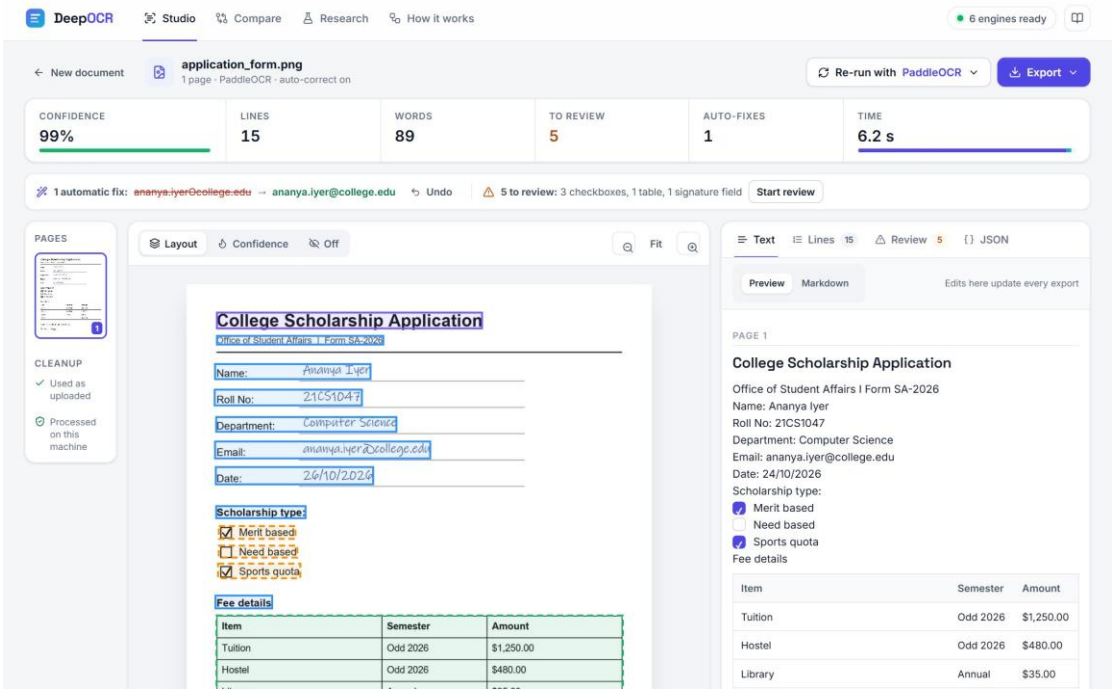


Figure 6.5: Layout analysis of an application form: headings, checkboxes, a ruled table and fields

6.10.4 Compare, Research and How It Works

The Compare page (Figure 6.6) runs several engines one after another on the same document. With ground-truth text, which is included for two of the samples, it scores each engine (CER, WER, time), marks the most accurate and the fastest, and highlights every misread or missing word.

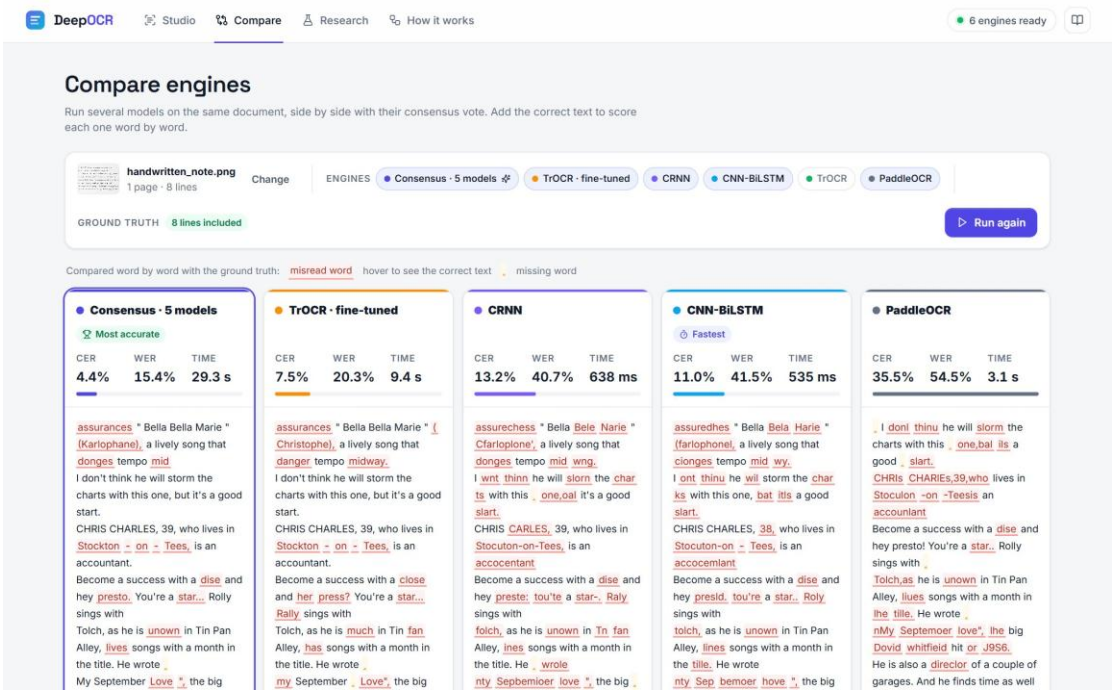


Figure 6.6: Compare page: five engines on the handwritten sample, scored word by word against the ground truth

The Research page (Figure 6.7) presents all experimental results interactively: headline numbers, the model comparison, the voting analysis, training and fine-tuning curves, robustness charts, sample predictions, the architectures and the datasets. The How it works page describes the eight pipeline stages.

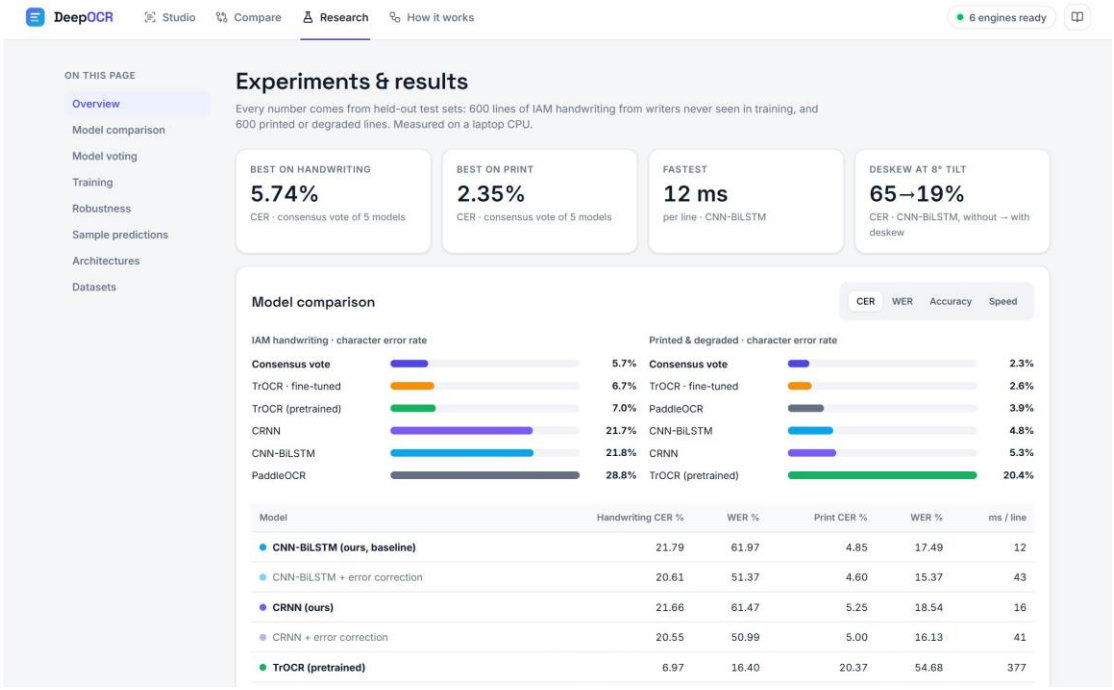


Figure 6.7: Research page with the measured results

6.10.5 Responsive Design

The layout adapts to tablets and phones (Figure 6.8). On narrow screens, a switch toggles between the document and the text, and export moves to a fixed bottom bar. Automated browser checks at widths of 1440, 768 and 390 pixels confirmed that no page scrolls sideways.

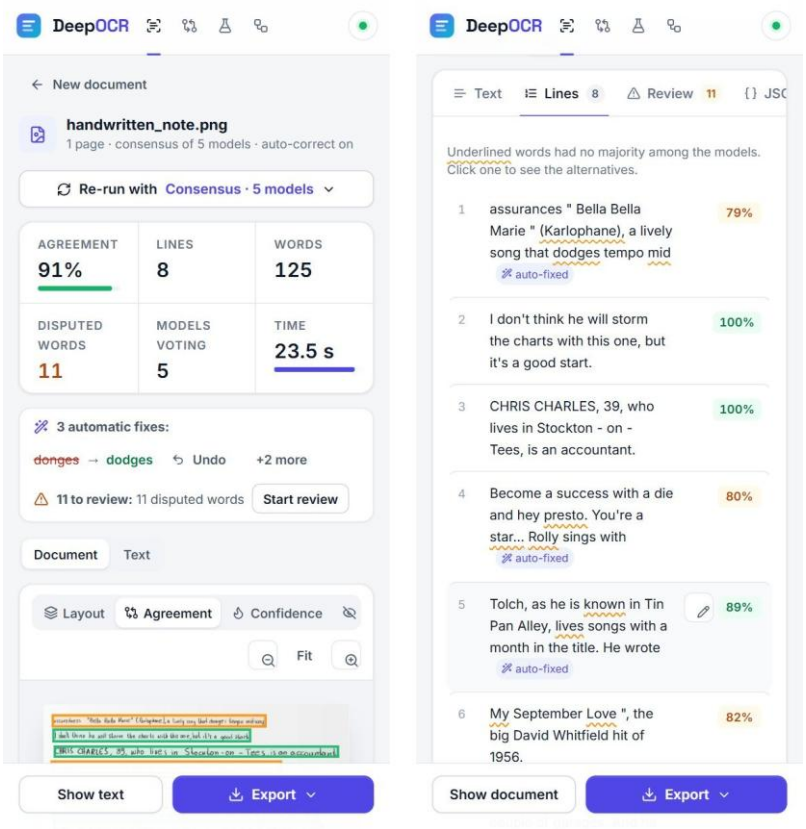


Figure 6.8: Results on a phone: document view (left) and recognised lines with disputed words (right)

6.11 Streamlit Research Interface

The proposal specified a Streamlit application. It is provided in `app/streamlit_app.py` and uses the same trained checkpoints and research pipeline. It lets a user upload an image, choose a model, view the detected regions and download the extracted text. The React studio was added for the richer review workflow described above.

7. Experiments and Results

7.1 Evaluation Metrics

The proposal's standard OCR metrics are used. The **Character Error Rate (CER)** is the Levenshtein distance between the predicted and the reference text at character level, divided by the length of the reference:

$$CER = (S + D + I) / N \times 100\% \tag{7.1}$$

where S, D and I are the numbers of substituted, deleted and inserted characters and N is the number of characters in the reference. The **Word Error Rate (WER)** is computed the same way over words:

$$WER = (Sw + Dw + Iw) / Nw \times 100\% \tag{7.2}$$

Lower CER and WER are better. Both are computed per line and averaged over the test lines. **Line accuracy** is the percentage of lines recognised exactly. **Inference time** is the CPU time per line, measured on the development laptop. Before scoring, predictions and references are normalised in the same way (Section 5.4); the raw-spacing CER is also stored in the result files.

7.2 Experimental Setup

Each engine read the same 600 IAM test lines (unseen writers) and the same 600 synthetic test lines, selected with seed 42 (`scripts/evaluate.py`). The line crops are given directly to each recogniser, so the comparison isolates recognition. The detector, layout and corrector are evaluated separately in Sections 7.6 and 7.10. All runs used the CPU of the development machine (Table 3.2).

7.3 Training Results

Figure 7.1 shows the validation CER and the CTC loss of the two models trained from scratch. Both converge quickly: the CER falls from about 80% after the first epoch to near 20% after four epochs and keeps improving slowly. The best validation CER is 11.8% for CNN-BiLSTM (epoch 11) and 12.2% for CRNN (epoch 8). The validation and training losses stay close together, so the augmentation prevented overfitting. Both curves were still descending when the time budget ended, so longer training on a GPU would improve the models further.

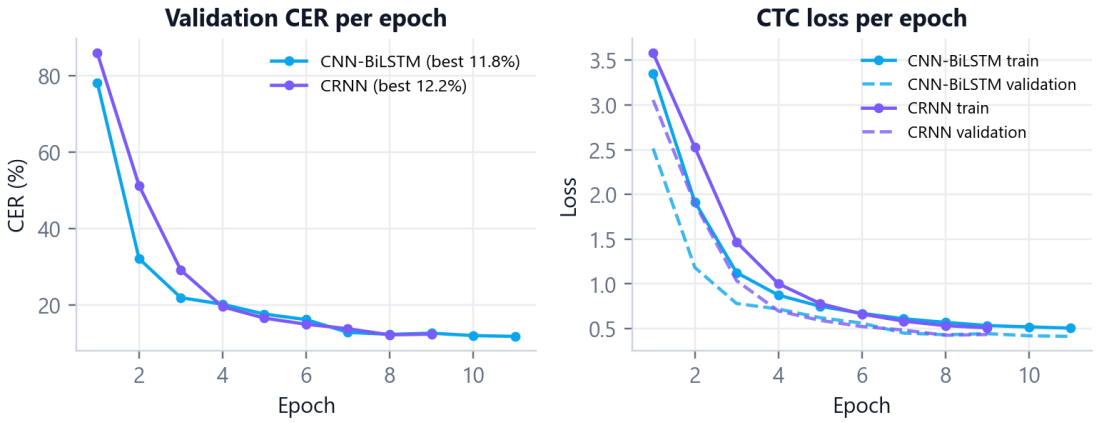


Figure 7.1: Training of the CTC models: validation CER (left) and CTC loss (right) per epoch

The validation CER (about 12%) is lower than the IAM test CER reported below (about 22%) for two reasons. The validation set mixes IAM and synthetic lines, and synthetic lines are easier. In addition, the IAM test writers are different people from the training writers, which is the hardest and most realistic setting.

7.4 Fine-Tuning Results

Figure 7.2 shows the fine-tuning of TrOCR. Its printed-text CER on the validation set fell from 23.0% to 3.7% within the first 75 steps and reached 2.5% after 300 steps. Its handwriting CER rose temporarily (from 3.3% to 7.4%) while the model adapted to the new data and then recovered to 4.3%. On the test sets, fine-tuning reduced the printed-text CER from 20.37% to 2.59% and left the handwriting CER essentially unchanged (6.97% → 6.73%). The fine-tuned model is therefore the best single model on both kinds of text.

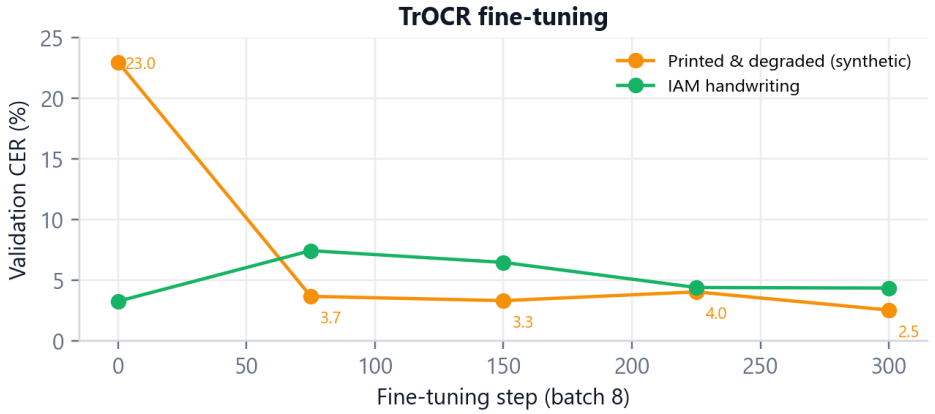


Figure 7.2: Validation CER of TrOCR during fine-tuning

7.5 Model Comparison

Table 7.1 is the main result of the project and completes the comparison table planned in the proposal. Figure 7.3 shows the same data.

Table 7.1: Recognition results on the held-out test sets (600 lines each; CER and WER in %, lower is better)

Model	Architecture	IAM CER	IAM WER	Print CER	Print WER	ms / line
CNN-BiLSTM (baseline)	CNN + BiLSTM + CTC	21.79	61.97	4.85	17.49	12
CRNN	VGG CNN + BiLSTM + CTC	21.66	61.47	5.25	18.54	16
TrOCR (pretrained)	ViT encoder + Transformer decoder	6.97	16.40	20.37	54.68	377
TrOCR (fine-tuned)	ViT encoder + Transformer decoder	6.73	16.79	2.59	10.81	415
PaddleOCR (reference)	PP-OCR recogniser	28.84	72.50	3.89	21.09	139
Consensus vote	**ROVER over all five**	**5.74**	**14.24**	**2.35**	**5.86**	**960**

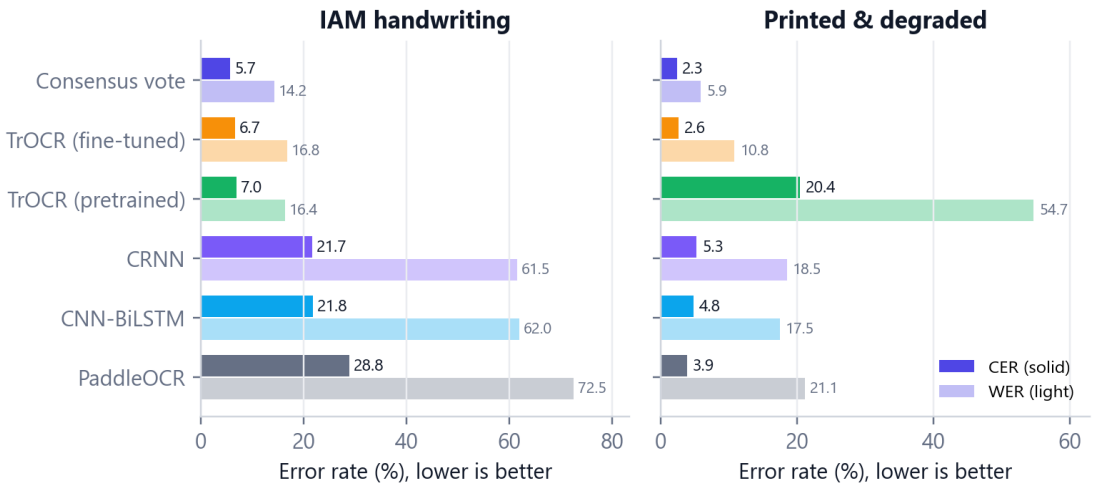


Figure 7.3: CER (solid) and WER (light) of each engine on handwriting and printed text

The main findings are:

- **Transformers lead on handwriting.** Both TrOCR models reach about 7% CER on handwriting from unseen writers, three times lower than the CTC models (about 22%). Large-scale pretraining and the language knowledge in the decoder make the difference.
- **The small CTC models are strong on print.** CNN-BiLSTM (4.85%) and CRNN (5.25%) nearly match PaddleOCR on printed text with only 2.4 to 2.8 million parameters, and they are the fastest engines (12 and 16 ms per line).

- **The baseline and CRNN perform almost equally.** The deeper VGG backbone of CRNN gives no clear gain at this data size, which agrees with Puigcerver's observation [8] that the recurrent layers matter more than the CNN depth.
- **Fine-tuning fixes TrOCR's weakness.** Without fine-tuning, TrOCR's printed-text WER is 54.68%; it often normalises capital letters and spelling. After fine-tuning it is 10.81%.
- **PaddleOCR is a print engine.** It has the highest handwriting error (28.84% CER) but good printed-text CER (3.89%).
- **The consensus vote is the most accurate engine** on both test sets: 5.74% CER and 14.24% WER on handwriting, and 2.35% CER and 5.86% WER on print. It is also the slowest (about 1 s per line), because every model runs.

Table 7.2: Line accuracy: percentage of test lines recognised exactly

Model	IAM handwriting	Printed & degraded
CNN-BiLSTM (baseline)	1.0	66.7
CRNN	0.5	63.2
TrOCR (pretrained)	40.7	53.8
TrOCR (fine-tuned)	37.8	84.3
PaddleOCR (reference)	1.5	66.2
Consensus vote	**42.3**	**89.5**

Line accuracy (Table 7.2) is a strict measure: one wrong character makes the whole line wrong. It shows the same order. The consensus reads 89.5% of printed lines and 42.3% of handwritten lines exactly.

7.6 Effect of Post-Processing

Table 7.3 shows the effect of the error corrector on the two CTC models, whose outputs contain the most spelling-like errors.

Table 7.3: Effect of the error-correction stage on the CTC models (CER and WER in %)

Model	Correction	IAM CER	IAM WER	Print CER	Print WER	ms / line
CNN-BiLSTM	without	21.79	61.97	4.85	17.49	12
CNN-BiLSTM	with correction	20.61	51.37	4.60	15.37	43
CRNN	without	21.66	61.47	5.25	18.54	16
CRNN	with correction	20.55	50.99	5.00	16.13	41

Correction lowers the handwriting WER by more than ten points (61.97% → 51.37% for CNN-BiLSTM) and the printed-text WER by about two points. The effect on CER is smaller, because a corrected word usually differs from the wrong one in only one or two characters. The cost is about 30 ms per line for the dictionary look-ups. The corrector is therefore enabled by default in the web app, and every change can be undone.

7.7 Consensus Voting

Table 7.4 compares the consensus vote with the best single model on each test set and with an *oracle* that knows the correct answer and picks the best model's reading for each line. The oracle is not achievable in practice; it shows how much room is left for any combination method.

Table 7.4: Consensus voting compared with the best single model (CER and WER in %; 2-fold cross-validated)

	IAM CER	IAM WER	Print CER	Print WER
Best single model	6.73	16.79	2.59	10.81
Consensus vote (cross-validated)	**5.74**	**14.24**	**2.35**	**5.86**
Oracle: best model chosen per line	4.27	12.19	0.49	2.31
Change: vote vs best single	-15%	-15%	-9%	-46%

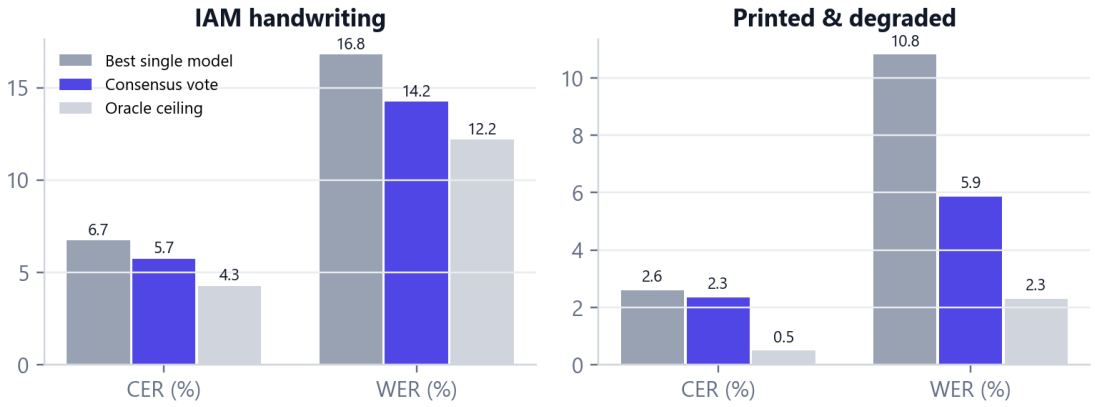


Figure 7.4: Best single model, consensus vote and oracle ceiling

Voting removes 15% of the character and word errors of the best single model on handwriting, and 46% of the word errors on printed text. On print the best single model already has a very low CER, so the vote mainly fixes whole words where one model failed, such as e-mail addresses that TrOCR distorts but the CTC models read correctly. The vote closes about 40% of the gap between the best single model and the oracle on

handwriting. The learned weights are sensible: the two TrOCR models, the most accurate on handwriting, get the highest weights, and CRNN, which adds the least independent information beside the similar CNN-BiLSTM, gets the lowest.

The support of each word is also useful on its own. In the handwritten demonstration page, 11 of 125 words had no majority, and they include most of the misread words on the page. A reviewer who checks only those 11 words finds the most likely mistakes without reading the whole page.

7.8 Robustness Study

The proposal lists real-world robustness testing as part of the novelty of the project. `scripts/robustness.py` re-read 80 IAM test lines under six controlled degradations at increasing strength (Figure 7.5, Table 7.5).

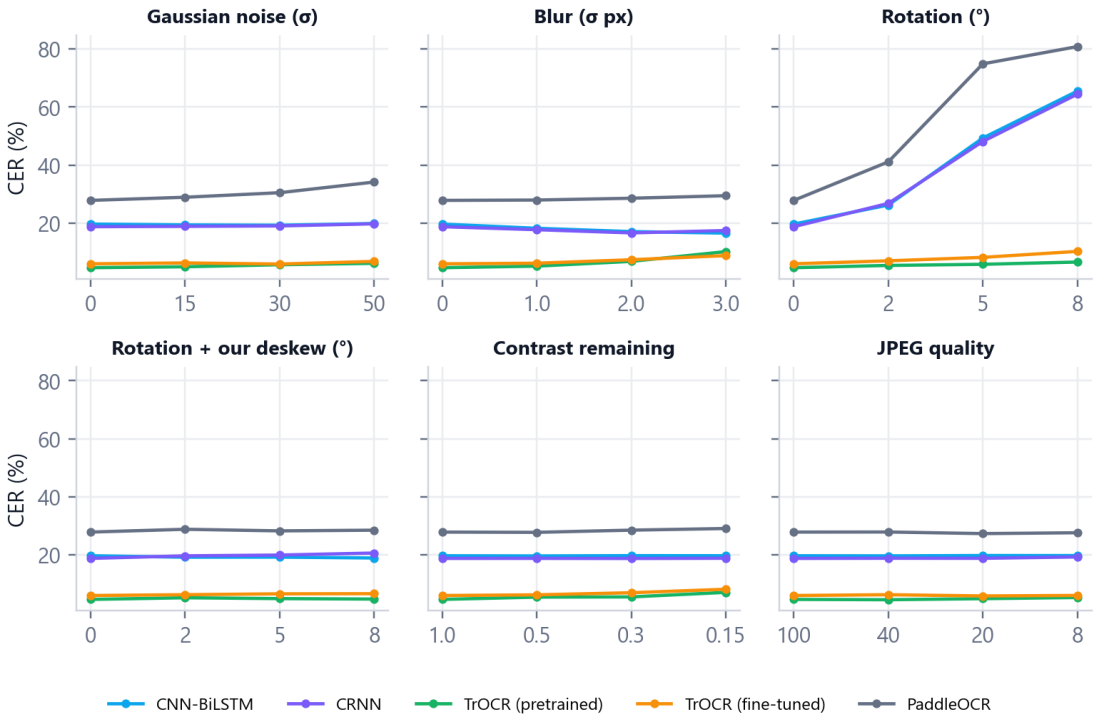


Figure 7.5: CER of each engine as each degradation gets stronger

Table 7.5: CER (%) at the strongest level of each degradation (80 IAM test lines)

Condition	CNN-BiLSTM	CRNN	TrOCR	TrOCR-ft	PaddleOCR
Clean (no degradation)	19.6	18.7	4.6	5.9	27.8
Gaussian noise, σ = 50	19.8	19.7	6.1	6.8	34.1
Blur, σ = 3 px	16.5	17.4	10.2	8.8	29.4
Rotation 8°	65.5	64.4	6.5	10.2	80.8

Condition	CNN-BiLSTM	CRNN	TrOCR	TrOCR-ft	PaddleOCR
Rotation 8° + our deskew	18.9	20.6	4.7	6.6	28.5
Contrast reduced to 15%	19.6	18.8	7.0	8.1	29.0
JPEG quality 8	19.7	19.2	5.3	6.0	27.6

- **Noise, low contrast and JPEG compression** barely affect the CTC models, which reflects the training augmentation and the shared line normalisation. Mild blur even lowers their error slightly, because it smooths ragged pen strokes.
- **Rotation is the main weakness.** An 8° tilt raises the CER of CNN-BiLSTM from 19.6% to 65.5% and of PaddleOCR from 27.8% to 80.8%.
- **Deskewing solves it.** With the project's deskew step in front, the CER at 8° returns to 18.9% (CNN-BiLSTM) and 28.5% (PaddleOCR), almost the clean values. This quantifies the value of the preprocessing stage.
- **TrOCR is the most robust to rotation** (6.6% at 8° without deskewing) but the most sensitive to strong blur (10.2% at $\sigma = 3$).

7.9 Accuracy Versus Speed

Figure 7.6 plots each engine's handwriting CER against its time per line. The engines form a clear trade-off. The CTC models are about 30 times faster than TrOCR and suit large volumes of printed text. Fine-tuned TrOCR offers the best balance for handwriting. The consensus vote gives the highest accuracy when the extra time is acceptable, for example for one important page. The web app shows each engine's measured error and speed so users can make this choice themselves.

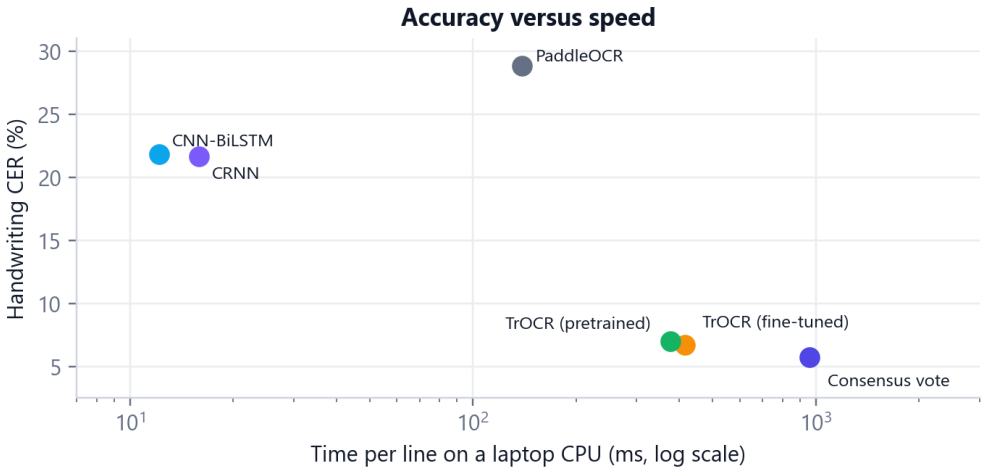


Figure 7.6: Accuracy versus speed of the engines (CPU, log scale)

7.10 End-to-End Results on a Document

The line-level results above assume perfect line crops. To test the complete pipeline (detection, recognition and layout), the handwritten sample page was processed with each engine through the Compare page with automatic correction turned off. This page contains eight IAM test lines, 125 words, from unseen writers (Table 7.6).

Table 7.6: End-to-end results on the handwritten sample page (8 lines, auto-correct off)

Engine	CER (%)	WER (%)	Time for the page
Consensus · 5 models	**4.4**	**15.4**	**29.3 s**
TrOCR (fine-tuned)	7.5	20.3	9.4 s
CNN-BiLSTM	11.0	41.5	0.54 s
CRNN	13.2	40.7	0.64 s
PaddleOCR	35.5	54.5	3.1 s

The order matches the line-level evaluation, and the consensus again gives the lowest error. Page times include detection and layout analysis, with all models already loaded in memory. PaddleOCR returned no text for the faint first line, which accounts for part of its high error.

7.11 Qualitative Results

Table 7.7 shows test lines and how the engines read them. TrOCR uses context to read whole words correctly, while the CTC models read character by character and produce near-misses such as “pecernce”. On structured printed text (e-mail addresses, upper-case phrases), the pretrained TrOCR “normalises” the text towards common English, which the fine-tuned model no longer does.

Table 7.7: Example predictions (errors in the CTC and TrOCR outputs are visible against the ground truth)

Model	Prediction
Ground truth	**of their presence. They just stared at him, turning**
CNN-BiLSTM	of their pecernce. They just stared at himn, twening
CRNN	of their pecence. They jiust stased ot him, trenin
TrOCR (fine-tuned)	of their presence. They just starred at him, turning
PaddleOCR	o heir pecence. They just stased ot him, tuning
Ground truth	**fatima.wilson@college.edu**
TrOCR (pretrained)	Fatima_wilson_ goodbye, edit
TrOCR (fine-tuned)	fatima. wilson@college.edu

Model	Prediction
CNN-BiLSTM	faulima.wilson@college.edr
Ground truth	**38 PER CENT**
TrOCR (pretrained)	38 per cent.
CRNN / TrOCR (fine-tuned)	38 PER CENT

7.12 Discussion

The experiments confirm the proposal's expectations and add one clear lesson. Transformer models capture long-range context and handwriting styles far better than CNN-BiLSTM models of the same era, as expected, but they need pretraining and are slow on a CPU. The small CTC models, trained from scratch in two hours, are practical for printed text. The most effective single improvement, however, did not come from a bigger model. It came from letting the existing models vote. Agreement between independent recognisers both reduces errors and tells the user where the remaining errors are likely to be, which is what matters in practice when a person must trust the output.

8. Testing

8.1 Testing Strategy

Testing was carried out at four levels: unit tests for individual functions (metrics, normalisation, voting, correction), integration tests for the models and the research pipeline, API tests that run the FastAPI service in-process, and frontend tests that render the React application in a simulated browser. Finally, the complete system was checked end to end in a real browser (Microsoft Edge, through Playwright) on desktop, tablet and phone screen sizes.

8.2 Automated Test Suites

Table 8.1: Automated tests (all passing)

Suite	File(s)	Tests	What is tested
Metrics	tests/test_metrics.py	16	CER, WER, accuracy, edge cases (empty text, normalisation)
Datasets and splits	tests/test_dataset.py, test_splits.py	16	Loading, charset encoding, writer-independent splits
Models and CTC	tests/test_models.py, test_ctc_utils.py	12	Output shapes, parameter counts, CTC decoding
Preprocessing	tests/test_preprocessing.py	9	Deskew angle, denoise, contrast, binarisation
Research pipeline	tests/test_pipeline.py, test_evaluate_script.py	8	End-to-end recognition and evaluation script
Consensus voting	tests/test_ensemble.py	5	Alignment, weighted votes, support, ties
Reference engine	tests/test_paddleocr_evaluation.py	3	PaddleOCR evaluation adapter
API	api/tests/test_api.py	7	Endpoints, errors, limits, health
Documents	api/tests/test_documents.py	6	PDF and image decoding, size and page limits
OCR engine	api/tests/test_ocr_engine.py	4	Detection and recognition wrapper
Pipeline options	api/tests/test_pipeline_options.py	18	Engines, jobs and progress, consensus words, corrector, skew
Layout	api/tests/test_structure.py	10	Headings, tables, checkboxes, signatures, grouping
Backend total	**pytest**	**114**	
Web app	web/src/App.test.tsx	10	Studio flow, disputed words, undo, export, errors, Compare, Research
Document model	web/src/lib/document.test.ts	7	Editing, word replacement, undo, review queue
Text utilities	web/src/lib/text.test.ts	4	CER/WER and diff helpers
Frontend total	**Vitest**	**21**	

All 135 tests pass. The type checker (`tsc`) and the production build of the web app also complete without errors.

8.3 Test Cases

Table 8.2: Selected system test cases

ID	Test case	Expected result	Result
TC1	Upload a PNG of handwriting with the consensus engine	Progress shows five models; result shows agreement and disputed words	Pass
TC2	Click a disputed word and choose an alternative	Word replaced; review count decreases; export contains the new word	Pass
TC3	Undo an automatic correction	Original word restored in the lines and exports	Pass
TC4	Upload a .txt file	Rejected with “Unsupported file type” before upload	Pass
TC5	Upload a 101 MB image	Rejected with a 100 MB limit message	Pass
TC6	Upload an invalid PDF	Server error 422 shown as a readable message	Pass
TC7	Send a second document while one is processing	HTTP 429; the client waits and retries automatically	Pass
TC8	Cancel a running job	Server stops the job; the app returns to the previous screen	Pass
TC9	Process the skewed noisy scan with the Scan preset	Skew of about 4° detected and corrected; text read in order	Pass
TC10	Process the application form	Title, checkboxes, ruled table and signature field reconstructed	Pass
TC11	Compare engines on a sample with ground truth	CER/WER per engine; most accurate and fastest marked	Pass
TC12	Open each page at 390 px width	No horizontal scrolling; document/text switch and bottom bar	Pass

8.4 Bugs Found and Fixed

Testing found several defects, which were fixed and covered by tests:

- Deskewing a noisy, unevenly lit scan returned −13° instead of −4°, because shadows dominated the projection profile. A flat-field ink mask fixed it.
- Tilted handwritten lines were merged into one line. Line grouping now uses each fragment's rotated geometry.
- The detector missed a faint handwritten line. Its box threshold was lowered to 0.5.
- The spelling corrector changed names (“Iyer” → “Yer”, “Tolch” → “Touch”). Capitalised words now accept only letter swaps.

- TrOCR fine-tuning did not converge (loss about 11.7) because of a wrong decoder start token. Fixing the token made the loss fall normally.
- On Windows, loading PaddlePaddle before PyTorch caused a DLL error (WinError 127). The import order was fixed.
- Comparing engines line by line misaligned the diff when an engine split or merged lines. A whole-document word alignment replaced it.

9. Conclusion and Future Work

9.1 Conclusion

This project developed DeepOCR, an intelligent deep-learning OCR system for handwritten and complex documents. It combines image preprocessing, DBNet text detection, deep-learning recognition, consensus voting, layout analysis and error correction in one pipeline, and presents the result in a web studio designed for review.

All three model families of the proposal were implemented and compared on the same held-out data. The CNN-BiLSTM baseline and the CRNN, trained from scratch on a laptop CPU, reach about 5% CER on printed text and 22% on handwriting from unseen writers, at 12 to 16 ms per line. The Transformer-based TrOCR, used through transfer learning and fine-tuned on the project's data, reaches 6.73% CER on handwriting and 2.59% on print. Combining all five recognisers with cross-validated ROVER voting gives the best results: 5.74% CER and 14.24% WER on handwriting, and 2.35% CER and 5.86% WER on print. The vote also identifies the words that need a human check. The robustness study shows that the preprocessing stage removes the main weakness of the models, rotation, and that the trained models tolerate noise, low contrast and compression.

9.2 Objectives Achieved

Table 9.1: Objectives of the proposal and how they were met

Objective	How it was achieved
1. OCR system for document images	Complete pipeline from PDF or image to structured text (Chapters 4 and 6)
2. Handwritten and printed text	Trained on IAM handwriting and synthetic print; 5.74% / 2.35% CER (Table 7.1)
3. Preprocess noisy, blurred, skewed images	Deskew, denoise, CLAHE, binarisation; robustness study (Section 7.8)
4. Detect and extract text regions	DBNet line polygons with perspective-rectified crops (Section 6.4)
5. Document layout analysis	Reading order, headings, tables, checkboxes, signature fields (Section 6.7)
6. Compare deep-learning models	CNN-BiLSTM, CRNN, TrOCR (zero-shot and fine-tuned), PaddleOCR, consensus (Section 7.5)
7. Standard OCR metrics	CER, WER, line accuracy and inference time on held-out test sets (Section 7.1)

Objective	How it was achieved
8. User-friendly interface	Web studio with review workflow, Compare and Research pages; Streamlit app (Section 6.10)

9.3 Limitations

- The CTC models were trained for about two hours on a CPU with reduced channel widths. A GPU, longer training, wider layers and a language model with beam search would lower their error further.
- TrOCR-small-handwritten was fine-tuned by Microsoft on IAM. Its IAM split may differ from the one used here, so its IAM score may be optimistic.
- Layout reconstruction is rule-based and has not been benchmarked against a labelled layout dataset; tables and checkboxes are therefore flagged for review.
- The consensus engine is two to three times slower than fine-tuned TrOCR because every model runs.
- Only English is supported.
- The default deployment has no user authentication or HTTPS; it is meant for local use or behind a secured proxy.

9.4 Future Work

- **Multilingual and Hindi recognition:** train on Devanagari datasets and add a script classifier, as listed in the proposal's future scope.
- **Learned layout analysis:** use models such as LayoutLM or DocTR for complex multi-column documents and forms.
- **Faster consensus:** run the models in parallel or on a GPU, and skip the vote when the first models already agree.
- **Language model decoding:** add beam search with a word-level language model to the CTC models.
- **Mathematical expressions and tables:** dedicated recognisers for equations and table structure.
- **Signature detection and document classification.**
- **Mobile and real-time use:** a mobile app with live camera OCR, using the compact CTC models.
- **Vision-language models:** use large multimodal models for question answering over documents, with the consensus vote as a check on their output.

References

[1] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[2] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.

[5] A. Graves, S. Fernández, F. Gomez and J. Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” in *Proc. 23rd International Conference on Machine Learning (ICML)*, 2006, pp. 369–376.

[6] A. Graves and J. Schmidhuber, “Offline handwriting recognition with multidimensional recurrent neural networks,” in *Advances in Neural Information Processing Systems 21 (NeurIPS)*, 2009, pp. 545–552.

[7] B. Shi, X. Bai and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[8] J. Puigcerver, “Are multidimensional recurrent layers really necessary for handwritten text recognition?” in *Proc. 14th International Conference on Document Analysis and Recognition (ICDAR)*, 2017, pp. 67–72.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[10] A. Dosovitskiy *et al.*, “An image is worth 16×16 words: Transformers for image recognition at scale,” in *Proc. International Conference on Learning Representations (ICLR)*, 2021.

[11] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles and H. Jégou, “Training data-efficient image transformers & distillation through attention,” in *Proc. 38th International Conference on Machine Learning (ICML)*, 2021.

[12] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li and F. Wei, “TrOCR: Transformer-based optical character recognition with pre-trained

models," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 37, 2023, pp. 13094–13102.

[13] X. Zhou, C. Yao, H. Wen, Y. Wang, S. Zhou, W. He and J. Liang, "EAST: An efficient and accurate scene text detector," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 5551–5560.

[14] Y. Baek, B. Lee, D. Han, S. Yun and H. Lee, "Character region awareness for text detection," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9365–9374.

[15] M. Liao, Z. Wan, C. Yao, K. Chen and X. Bai, "Real-time scene text detection with differentiable binarization," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 34, 2020, pp. 11474–11481.

[16] Y. Du *et al.*, "PP-OCR: A practical ultra lightweight OCR system," *arXiv preprint* arXiv:2009.09941, 2020.

[17] G. Nagy, S. Seth and M. Viswanathan, "A prototype document image analysis system for technical journals," *Computer*, vol. 25, no. 7, pp. 10–22, 1992.

[18] X. Zhong, J. Tang and A. Jimeno Yepes, "PubLayNet: Largest dataset ever for document layout analysis," in *Proc. International Conference on Document Analysis and Recognition (ICDAR)*, 2019, pp. 1015–1022.

[19] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," in *Proc. 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2020, pp. 1192–1200.

[20] J. G. Fiscus, "A post-processing system to yield reduced word error rates: Recognizer output voting error reduction (ROVER)," in *Proc. IEEE Workshop on Automatic Speech Recognition and Understanding*, 1997, pp. 347–354.

[21] W. B. Lund and E. K. Ringger, "Improving optical character recognition through efficient multiple system alignment," in *Proc. 9th ACM/IEEE-CS Joint Conference on Digital Libraries (JCDL)*, 2009, pp. 231–240.

[22] K. Kukich, "Techniques for automatically correcting words in text," *ACM Computing Surveys*, vol. 24, no. 4, pp. 377–439, 1992.

[23] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[24] U.-V. Marti and H. Bunke, "The IAM-database: An English sentence database for offline handwriting recognition," *International Journal on Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.

[25] K. Zuiderveld, “Contrast limited adaptive histogram equalization,” in *Graphics Gems IV*, P. S. Heckbert, Ed. San Diego: Academic Press, 1994, pp. 474–485.

[26] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[27] T. Wolf *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proc. Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.

[28] G. Bradski, “The OpenCV library,” *Dr. Dobb's Journal of Software Tools*, 2000.

Appendix A: Installation and Usage

Quick start on Windows. Double-click `start.bat` in the project folder, or run `.\start.ps1` in PowerShell. The launcher installs missing Python packages, builds the web app once (Node.js 20 or later is needed for that build), starts the server and opens `http://127.0.0.1:8000`. The API documentation is at `http://127.0.0.1:8000/api/docs`.

Manual start

```
python -m pip install -r web-requirements.txt
cd web
npm ci
npm run build
cd ..
python -m uvicorn api.main:app --host 127.0.0.1 --port 8000
```

Reproducing the experiments

```
python -m pip install -r requirements.txt
python scripts/download_iam_hf.py                    # IAM lines
python scripts/prepare_dataset.py --dataset synthetic --n_samples 6500
python scripts/train.py --model cnn_bilstm --epochs 11 --max_minutes 170
python scripts/train.py --model crnn --epochs 9 --max_minutes 170
python scripts/finetune_trocr.py --steps 300         # transfer learning
python scripts/evaluate.py --all --limit 600         # held-out test sets
python scripts/ensemble_eval.py                      # voting weights
python scripts/robustness.py --lines 80              # degradation study
python scripts/make_samples.py                       # demo documents
```

Running the tests

```
python -m pytest -q          # 114 backend tests
cd web
npm test                     # 21 frontend tests
npm run build                # type check and production build
```

Docker. Copy `.env.example` to `.env` and run `docker compose up --build -d`; the app is then served at `http://localhost:8080`.

Streamlit interface. `python -m streamlit run app/streamlit_app.py --server.address 127.0.0.1`.

Appendix B: Project Structure

```
DeepOCR/
├── api/              FastAPI service: decoding, cleanup, engines, jobs, layout
├── app/              Streamlit research interface
├── checkpoints/      Trained weights of each model and the fine-tuned TrOCR
├── configs/          Dataset, model and training configuration (YAML)
├── data/             Prepared datasets (generated by the scripts)
├── datasets/         Loaders, augmentation, shared line normalisation, splits
├── detection/        OpenCV MSER text detector (fallback)
├── evaluation/       CER / WER / accuracy metrics
├── inference/        End-to-end research pipeline
├── layout/           Research layout analyser
├── models/           CNN-BiLSTM, CRNN, Transformer OCR, recogniser wrappers
├── postprocessing/   Text corrector and ROVER consensus voting
├── preprocessing/    Image preprocessing operations
├── report/           This report, its figures and the scripts that build it
├── results/          Evaluation JSON/CSV, plots, training histories
├── samples/          Demo documents with ground truth
├── scripts/          Data, training, fine-tuning, evaluation, robustness
├── tests/            Research pipeline tests
├── training/         Trainer and CTC utilities
├── web/              React + TypeScript web studio
├── start.ps1 / .bat  One-click launcher
└── docker-compose.yml
```

Appendix C: Example API Response

An abridged response of `POST /api/v1/ocr` for the handwritten sample read with the consensus engine and auto-correct on. The background job endpoint returns the same object in its `result` field. Each block carries the per-word votes used by the review interface; the values below are from an actual run.

```
{
  "filename": "handwritten_note.png", "engine": "ensemble",
  "inference_ms": 19805.4,
  "stats": { "pages": 1, "lines": 8, "words": 125, "characters": 622,
             "mean_confidence": 0.8866, "corrections": 3 },
  "timings": { "decode": 78, "preprocess": 0, "recognition": 19569,
               "layout": 44, "postprocess": 43, "total": 19805 },
  "pages": [{
    "page_number": 1, "width": 2330, "height": 1112,
    "blocks": [ ...,
      {
        "kind": "text",
        "text": "Tolch, as he is known in Tin Pan Alley, lives songs with ...",
        "polygon": [[75, 558], [2107, 558], [2107, 643], [75, 643]],
        "confidence": 0.923, "review_required": false,
        "corrections": [{ "original": "unown", "corrected": "known",
                          "kind": "spelling", "position": 16 }],
        "words": [ ...,
          { "text": "Pan", "support": 0.656,
            "sources": ["trocr", "paddle", "cnn_bilstm"],
            "alternatives": { "fan": 0.344 } },
          { "text": "lives", "support": 0.281, "sources": ["trocr"],
            "alternatives": { "has": 0.255, "liues": 0.193,
                              "lines": 0.182, "ines": 0.089 } },
          ... ]
      }, ... ]
  }]
}
```

Here “Pan” won with 65.6% of the vote and is accepted, while “lives” (the correct word is “likes”) won only 28.1%, so it is marked as disputed and offered for review with its alternatives.